

AARHUS UNIVERSITET

SON

SEMESTERPROJEKT 2



2. Semester - Gruppe 8

Afleveringsdato: 12/12/2025

Studie Nr.:	Navn:	Studieretning
202410102	Mads Dessau Arp Posborg	ECE - SW 2. semester
202410114	Alexander Drexel Hougaard	ECE - SW 2. semester
202411104	Gustav Emil Wester	ECE - SW 2. semester
202308201	Malthe Winther	ECE - SW 2. semester
202410110	Oskar Jentzsch Seeberg	ECE - SW 2. semester
202410107	Viktor Kalhøj Sønderup Laursen	ECE - SW 2. semester

Vejleder: Rasmus Vorup (RV)



Resumé

Denne rapport beskriver udviklingsprocessen af det solcelledrevne lampesystem *Sol Om Natten* (SON). Systemet kombinerer målinger fra en lyssensor og en bevægelsessensor med brugerdefinerede indstillinger fra en GUI for at levere fleksibel og energieffektiv udendørs belysning. SON drives af et integreret PowerManagement-system, som muliggør drift uden ekstern strømforsyning.

Rapporten gennemgår projektets udvikling trin for trin, fra systembeskrivelse, kravspecifikation og teknisk analyse til arkitektur, design og implementering. Arkitektur og design fastlægges gennem UML- og SysML-diagrammer, herunder klassediagrammer, sekvensdiagrammer og statemachines, som tilsammen skaber en klar strukturel og adfærdsmæssig beskrivelse af systemet og sikrer sporbarhed mellem krav og implementering.

Systemets moduler testes individuelt gennem modultest, hvorefter en samlet integrationstest verificerer kommunikationen mellem sensorer, styringslogik og GUI'en. Afslutningsvis udføres en accepttest baseret på de opstillede funktionelle og ikke-funktionelle krav.

På baggrund af accepttesten vurderes projektet som en succes. SON opfylder størstedelen af de definerede krav, herunder begge driftsmodes, lysintensitetsstyring, Bluetooth-kommunikation og energistyring. Den iterative udviklingsproces har understøttet et robust forløb og resulteret i en fuldt fungerende prototype.

Abstract

This report describes the development process of the solar-powered lighting system *Sol Om Natten* (SON). The system combines measurements from a light sensor and a motion sensor with user-defined settings from a GUI to provide flexible and energy-efficient outdoor illumination. SON is powered by an integrated PowerManagement system, enabling operation without external power supply.

The report outlines the project step by step from system description, requirements specification, and technical analysis to architecture, design, and implementation. The architecture and design are established through UML and SysML diagrams, including class diagrams, sequence diagrams, and state machines, which together provide a clear structural and behavioral representation of the system and ensure traceability between requirements and implementation.

The system's modules are individually verified through module tests, followed by an integration test that validates communication between sensors, control logic, and the GUI. Finally, an acceptance test is performed based on the defined functional and non-functional requirements.

Based on the acceptance test, the project is considered successful. SON fulfills the majority of the specified requirements, including both operational modes, light-intensity control, Bluetooth communication, and power management. The iterative development approach supported a robust workflow and resulted in a fully functional prototype.

Indholdsfortegnelse

1.	Indledning	1
1.1	Problemformulering	1
1.2	Systembeskrivelse	2
2.	Metode og proces.....	4
2.1	Metode	4
3.	Kravspecifikation	9
3.1	Funktionelle krav.....	9
3.2	Ikke-Funktionelle krav.....	13
4.	Teknisk Analyse	14
5.	Systemarkitektur.....	16
5.1	Hardwarearkitektur.....	17
5.2	Softwarearkitektur	20
6.	Hardware Design og Implementering	24
6.1	PowerManagement.....	24
6.2	LED	26
6.3	Lightsensor	28
6.4	MotionSensor.....	30
6.5	Bluetooth	31
6.6	Samlet hardware-overblik	33
7.	Software Design og Implementering	36
7.1	LED	36
7.2	LightDriver.....	38
7.3	MotionDriver.....	40
7.4	System.....	41
7.5	Bluetooth	46
7.6	GUI.....	48
8.	3D printet kabinet	50
9.	Modul- og integrationstests	51
9.1	Modultest for PowerManagement	51
9.2	Modultest for System-klassen	52

9.3	<i>Modultest for GUI</i>	53
9.4	<i>Modultest for Bluetooth + GUI-Bluetooth-Kommunikation</i>	54
9.5	<i>Integrationstest</i>	55
10.	Resultater	56
11.	Diskussion og fremtidigt arbejde	59
12.	Konklusion	61
13.	Individuelle konklusioner	62
14.	Bilagsliste	65
15.	Bibliografi	66

Figurer

Figur 1 - Digital skitse - 1. Lyssensor, 2. Solcelle, 3. Solcellecontroller, 4. Batteri, 5. Lampe, 6. Mikrocontroller, 7. GUI, 8. Bevægelsessensor.	3
Figur 2 - ECE-modellen.....	4
Figur 3 - Projektets tidsplan.....	8
Figur 4 - Aktør-kontekst-diagram.....	9
Figur 5 - Use Case Diagram.....	10
Figur 6 - Domænemodel for SON	16
Figur 7 - BDD for SON	17
Figur 8 - IBD for SON	18
Figur 9 - Overordnet klassediagram	20
Figur 10 - Klassediagram for UC1.....	21
Figur 11 - Sekvensdiagram for step 1-4 i UC1	21
Figur 12 - Sekvensdiagram for step 5 i UC1	22
Figur 13 - Overordnet klassediagram med funktioner	22
Figur 14 - PowerManagement-kredsløb	24
Figur 15 - Implementering af 5V fordeler	25
Figur 16 - Implementering af PowerManagemen	25
Figur 17 - Kredsløb til LED.....	26
Figur 18 - Implementering af LED	27
Figur 19 - Kredsløbsdiagram til lyssensor	28
Figur 20 - Implementeret lyssensor kredsløb.....	29
Figur 21 - Kredsløbsdiagram til bevægelsessensor	30
Figur 22 - Implementeret bevægelsessensor kredsløb	30
Figur 23 - Kredsløbsdiagram til Bluetooth	31

Figur 24 - Implementering af Bluetooth-kredsløb	32
Figur 25 - Kredsløbsdiagram til SON	33
Figur 26 - Billede af Arduino-forbindelserne	34
Figur 27 - LEDDriver klassediagram	36
Figur 28 - LightDriver klassediagram.....	38
Figur 29 - MotionDriver klassediagram	40
Figur 30 - Systemkode UML- diagram.....	41
Figur 31 - Simpel Switch-case	43
Figur 32 - update() - Automatic Mode.....	44
Figur 33 - update() - Motion Mode.....	44
Figur 34 - GUI Klassediagram.....	48
Figur 35 - CAD Model af systemet SON.....	50
Figur 36 - Resultat af accepttest.....	57
Figur 37 - Billede af det samlede produkt	58

Tabeller

Tabel 1 - Ordforklaring.....	vii
Tabel 2 - Tabel over arbejdsfordeling.....	7
Tabel 3 - Aktør-kontekst.....	10
Tabel 4 - Fully Dressed Use Case for UC1.....	11
Tabel 5 - Prioritering af funktionelle krav	12
Tabel 6 - Tabel over ikke-funktionelle krav prioriteret med MoSCoW	13
Tabel 7 - Teknisk Analyse Tabeloversigt	15
Tabel 8 - Blokbetegnelse for BDD	19
Tabel 9 - Funktionsbetegnelse af I2C funktioner	38
Tabel 10 - UART-kommunikation funktioner og variabler.....	46
Tabel 11 - GUI Klassediagram	48
Tabel 12 – Kommandobetegnelser	49
Tabel 13 - Modultest af PowerManagement.....	51
Tabel 14 - Modultest af System-klassen	52
Tabel 15 - Modultest af GUI.....	53
Tabel 16 - Modultest af Bluetooth + GUI.....	54
Tabel 17 - Tabel over resultater	56
Tabel 18 - Tabel over accepttest	57

Forord

Denne projektrapport er udarbejdet af seks 2. semester softwareteknologistuderende ved Aarhus Universitet, som en del af projektforløb SW2PRJ2-03.

Projektet er vejledt af Rasmus Vorup, som vi gerne vil takke for faglig sparring og konstruktiv feedback gennem hele forløbet.

Rapporten består af 74 sider med 72000 tegn.

Rapporten er afleveret 12/12/2025 og bedømmes ved intern censur til et mundtlig forsvar.

Ordforklaring

Ord	Forklaring
SON	Forkortelse for Sol Om Natten, og betegner projektets samlede løsning og omfatter både hardware og software som én integreret helhed.
SYSTEMET	Betegner den interne software, system-klassen, der styrer mikrocontrollerens funktionalitet.
Qt	Et C++ framework til programmering af GUI. Herudover understøtter det kommunikation imellem GUI og systemet gennem COM-porte.
Automatic Mode	Automatic Mode betyder at SON's lyssensor er aktiv. Her tændes lampen kun når lyssensoren måler en værdi lavere end den fastsatte lux grænse. Skrevet på engelsk for at sikre konsistens med diagrammer og kodning.
Motion Mode	Motion Mode hentyder til at SON's bevægelsessensor er aktiv. Her tændes systemet kun når bevægelsessensoren aktiveres. Skrevet på engelsk for at sikre konsistens med diagrammer og kodning.
Timer	Timer hentyder til tiden, det tager, fra en bevægelse detekteres, til lampen går inaktiv. Timerens værdi ligger mellem 5 og 120 sekunder. Betegnelsen skrives med stort <i>T</i> for at markere, at det er en navngiven funktion/parameter i systemet og ikke det almindelige danske ord <i>timer</i> .
Lysintensitet	Lysintensiteten angiver styrken af LED'ernes lysudsendelse, når SON er tændt og aktiv. Intensiteten justeres på en skala fra 0 til 100.
Mikrocontroller	I projektet anvendes en Arduino Mega 2560, som håndterer GPIO-styring, kommunikation og realtidsbehandling af input fra sensorer samt styring af aktuatorer.
PowerManagement	PowerManagement er en samlet hardwareblok, der håndterer forsyningen til SON. Blokken omfatter batteri, solcelle, solcellecontroller og spændingsregulator, og sørger for at konvertere, regulere og distribuere den nødvendige 5V-forsyning til systemets komponenter. Navnet er skrevet som ét ord for at understrege, at der er tale om en samlet hardwarekomponent. Refereres også til som PM.
Inaktiv	SON går i inaktiv tilstand, når GUI viser "on" men lysintensiteten er 0.
UC	Forkortelse for Use Case, bruges når der refereres til en bestemt Use Case.

Tabel 1 - Ordforklaring

1. Indledning

Global opvarmning er et af de største problemer vi som mennesker står overfor lige nu. Polerne smelter, havene stiger og biodiversiteten mindskes. For at stoppe denne udvikling, så har mange af verdens lande indgået i Parisaftalen, i et håb om at formindske CO₂ udledningen inden næste årti.

Som det næste årti nærmer sig, så leder den moderne forbrugerisme os længere og længere væk fra en CO₂-venlig verden¹. Restriktioner eller totalforbydning af visse industrier vil blot skabe problemer og møde modstand, da den moderne økonomi er dybt afhængig af et konstant, men energitunt, forbrug. Derfor er en stor del af løsningen at finde smarte og energieffektive alternativer, der kan integreres problemfrit i hverdagen uden at kræve store adfærdsændringer.

1.1 Problemformulering

Hvordan kan vi designe og udvikle en solcelledrevet lampe, der kombinerer energieffektivitet med fleksibel brugerinteraktion?

Systemets formål er at optimere energiforbruget gennem opladning og afladning af et batteri, samt registrere omgivelsernes lysniveau, som f.eks. solopgang og solnedgang, for at automatisere oplysning. Desuden skal systemet kunne operere i en bevægelsesbaseret driftsmode, der registrerer bevægelse - altså brugerens ankomst og afrejse. Samtidig skal brugeren have mulighed for at regulere lysintensitet på en GUI som er forbundet til lampen.

Formålet er at reducere afhængigheden af ekstern strømforsyning og unødigt kabelføring, samtidig med, at systemet tilbyder større fleksibilitet end andre tilsvarende produkter.

¹ Garric, Juli 2025, "Climate: What does it mean if we can no longer limit warming to 1.5°C?"

1.2 Systembeskrivelse

SON er en solcelledrevet lampe, der kan give fleksibel og energieffektiv belysning til udendørs brug. I rapporten og bilag vil der blive refereret til SON, som er navnet for vores samlede projekt, se rapportens ordforklaring.

Overordnet set skal SON kunne operere i to forskellige Modes:

1. **Automatic Mode**, hvor lampen styres af input fra en lyssensor, og lampen tændes når det bliver mørkt.
2. **Motion Mode**, hvor lampen styres af en bevægelsessensor, og lampen tændes når der detekteres bevægelse.

Derudover er der følgende indstillinger, som kan indstilles for lampen:

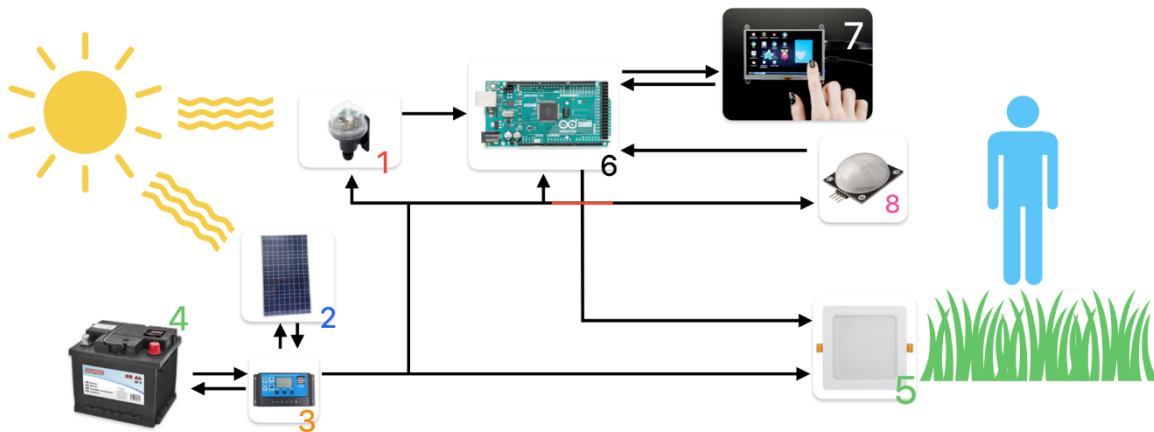
1. **Tænd/Sluk**. SON vil som udgangspunkt altid være tændt, og forbindelsen mellem GUI og Arduino vil altid være aktiv, alligevel kan lampen slukkes igennem GUI'en. Dette betyder at lampen stopper med at lyse, men alle indstillinger gemmes, og er aktive idet lampen tændes igen.
2. **Lysintensitet** er et mål for lampens lysintensitet og går fra 0-100%. Denne indstilling er uafhængig af lampens Mode.
3. **Timer** påvirker kun lampens drift når SON operer i Motion Mode. Timer er et mål for den mængde af tid lampen er tændt efter en bevægelse detekteres af bevægelsessensoren. Denne indstilling går fra 5-120 sekunder og kan indstilles i 1-sekunds intervaller.

SON består overordnet set af følgende 6 logiske funktionsblokke. Nedenstående er der en forklaring af hver af disse logiske funktionsblokke samt en digital skitse af SON's opbygning.

1. **PowerManagement**, som omfatter **solcellepanelet**, **solcellecontrolleren**, **spændingsregulator** og **batteriet**. Denne del står for at opsamle, lagre og distribuere energi til resten af systemet med undtagelse af Brugergrænsefladen, som kører lokalt på brugerens enhed.
2. **Mikrocontroller**, der håndterer data fra sensorer, styrer lyset og kommunikerer med Brugergrænsefladen.
3. **Lyssensor**, der måler omgivelsernes lysniveau brugt til styring af Automatic Mode.
4. **Bevægelsessensor**, der registrerer bevægelse brugt til styring af Motion Mode.

5. **Lampe**, der kan reguleres i intensitet.
6. **Brugergrænseflade**, som muliggør manuel styring af lysintensitet og systemtilstand via en **kommunikationsprotokol** til en GUI-applikation. GUI-applikationen skal kunne køre på et Windows-system, så kommunikationen med enheden kan foregå via computerens COM-porte.

Nedenstående er digital skitse over SON, der viser relationerne mellem de forskellige systemdele.



Figur 1 - Digital skitse - 1. Lyssensor, 2. Solcelle, 3. Solcellecontroller, 4. Batteri, 5. Lampe, 6. Mikrocontroller, 7. GUI, 8. Bevægelsessensor.

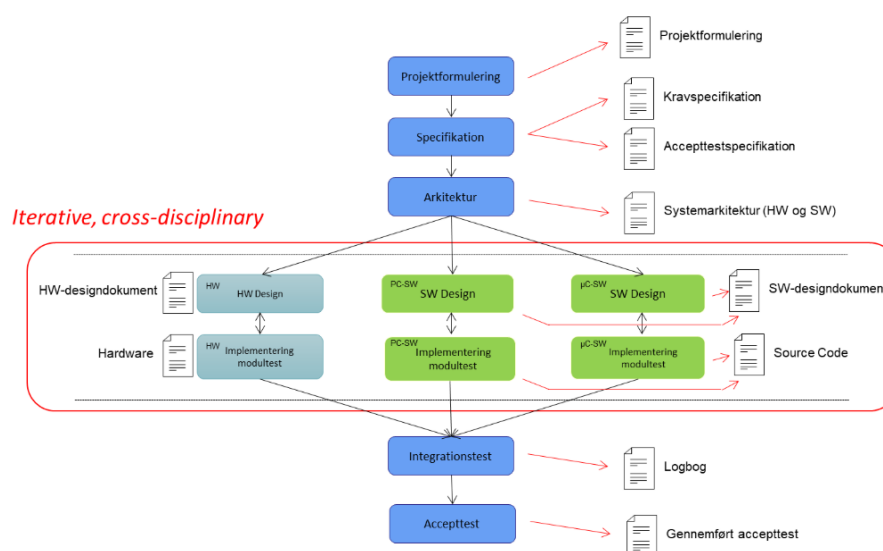
2. Metode og proces

I dette afsnit beskrives arbejdsprocessen for projektet samt de metoder, der er anvendt til at færdiggøre det. Afsnittet vil hovedsageligt omhandle ECE-modellen og detaljer om vores arbejdsfordeling og proces.

2.1 Metode

ECE-modellen er en udviklingsproces inddelt i tre faser.

1. I den indledende fase lægges fokus på planlægning. Den indeholder projektformulering, kravspecifikation, accepttestspecifikation og arkitektur, og udgør alt det forarbejde, der ligger forud for design, implementering og tests. Gruppen har i fællesskab udarbejdet en projektformulering, kravspecifikation, accepttestspecifikation og systemarkitektur.
2. I designfasen blev de løsninger, der var planlagt i den indledende fase, konkretiseret og implementeret som separate moduler. Her blev gruppen opdelt i arbejdsgrupper, der hver var ansvarlig for et eller flere af SON's funktioner.
3. Til sidst kommer integrations- og afslutningsfasen, hvor SON's delelementer blev samlet, og funktionaliteten samt samspillet mellem modulerne blev testet med det formål at opfylde kravene, defineret i kravspecifikationen.



Figur 2 - ECE-modellen²

² ICE Lektion 1

Projektmodellen har muliggjort en strømlinet udviklingsproces. En stor fordel ved modellen er, at den sikrer en tydelig struktur gennem hele projektforsløbet. Ved at opdele arbejdet i adskilte faser har gruppen haft mulighed for først at skabe et solidt fundament gennem planlægning og afklaringer, før design og implementering blev igangsat. Dette har reduceret risikoen for misforståelser og omformuleringer senere i processen.

2.1.1 Modelkonventioner

Sammenhængen mellem modeller og implementering er sikret igennem en række fastlagte konventioner for navngivning og struktur i alle SysML og UML-diagrammer.

❖ Sprog

Alle blokke, signaler, porte og forbindelser i modellerne på tværs af hardware- og softwarearkitektur samt designfasen navngives på engelsk for at sikre ensartethed mellem model og kode.

❖ Navngivning

Signalnavne benytter sig af samme princip som koden, altså camelCase for signaler og PascalCase for blokke og komponenter.

❖ Overensstemmelse

Der er tilstræbt fuld overensstemmelse mellem modellernes indhold og de tilsvarende variable og funktioner i implementeringen, således, at ændringer det ene sted kan spores direkte til det andet sted.

2.1.2 Standarder og konventioner for kode

For at sikre ensartethed, læsbarhed og let vedligeholdelse er der tilstræbt fælles standarder på tværs af koden, der følges igennem hele udviklingens processen.

❖ Navngivning

Variabler og funktioner benytter camelCase, mens klasser og strukturer benytter PascalCase.

❖ Sprogvalg

Al kode, både variabler, funktioner og kommentarer, skrives på engelsk for at opretholde en international standard og sammenhæng mellem model og navngivning i diagrammer.

2.1.3 Udviklingsmetode og værktøjer

Projektet er udarbejdet med en model- og diagrambaseret udviklingsmetode, hvor hardwarearkitekturen først bliver beskrevet ved hjælp af SysML-diagrammer, herunder Block Definition Diagrams (BDD) og Internal Block Diagram (IBD). Disse diagrammer opfølges med en detaljeret blok- og signalbeskrivelse. Dette er med henblik på at give et klart indblik i SON's hardwarearkitektur inden implementeringen. Softwarearkitekturen beskrives herefter ved brug af en domænemodel, og derefter ved hjælp af applikationsmodeller, med dertilhørende klassediagrammer, sekvensdiagrammer og eventuelle state-machine diagrammer for hver enkelt Use Case, beskrevet i kravspecifikationen. Der er fokus på sporbarhed mellem modeller, kode og signalnavne således at det alt sammen afspejles direkte i implementeringen.

Værktøjer som OneDrive, GitLab og Draw.io har været centrale i håndteringen af dokumentation, versionskontrol og visuel planlægning. De ugentlige vejledermøder har sikret, at projektet løbende blev justeret og målrettet undervejs.

2.1.4 Arbejdsproces

Arbejdsfordelingen har ændret sig undervejs i projektets tre faser. I den indledende fase arbejdede gruppen i fællesskab om forarbejdet, herunder projektformulering, kravspecifikation og systemarkitektur.

I designfasen blev gruppen opdelt i tre mindre grupper af to personer, der hver især havde ansvar for henholdsvis GUI, hardwarekomponenter og drivere og Bluetooth-kommunikation. De mindre grupper arbejdede dog ikke isoleret, da det var nødvendigt at sikre, at kommunikationen mellem de forskellige moduler fungerede korrekt. For eksempel skulle GUI og systemet kunne sende og modtage signaler via Bluetooth, hvilket krævede tæt koordinering mellem de involverede grupper.

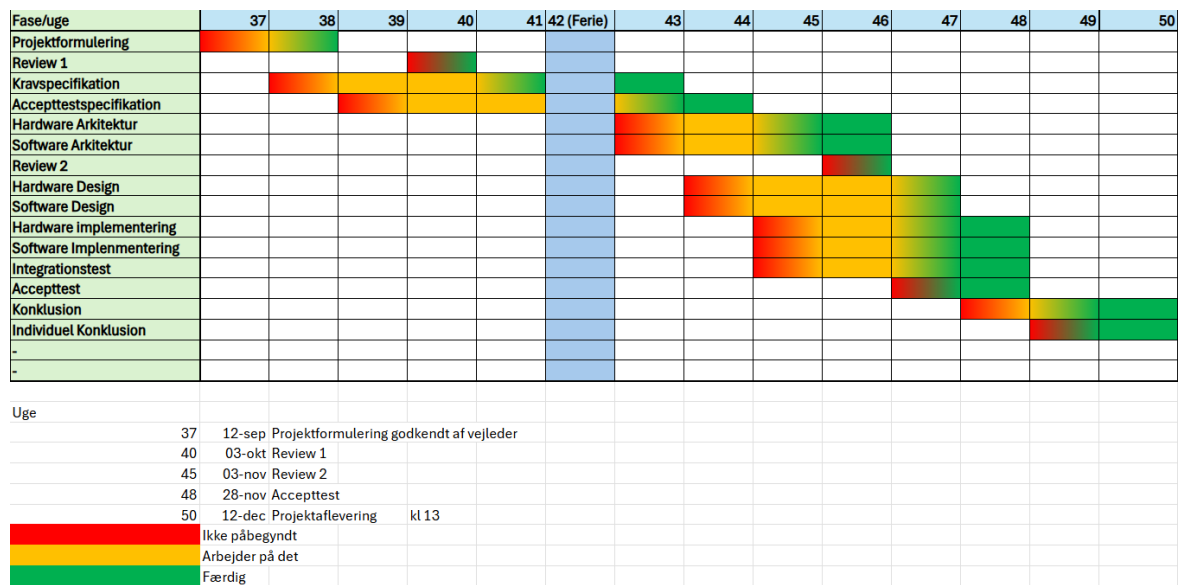
Denne arbejdsfordeling har fungeret godt. Den tydelige opdeling af ansvarsområder gjorde det muligt at arbejde effektivt og målrettet, samtidig med at specialisering inden for hvert modul blev fremmet. Den løbende kommunikation mellem grupperne samt den grundige indledende fase med nøje udarbejdede diagrammer sikrede, at integrationen af SON's moduler mere eller mindre problemfrit. Denne arbejdsfordeling er vist i nedenstående Tabel:

2.1.5 Arbejdsfordeling

Område	Fælles	MW	OJS	ADH	MDAP	GEW	VKS
Resume	○						
Forord	○						
Indledning	○						
Metode og proces		●			●	●	
Problemformulering	○						
Kravspecifikation	○						
Accepttestspecifikation	○						
Teknisk Analyse			●				
Systemarkitektur	○						
HW-design og HW-implementering							
- PowerManagement		●	●				
- LED		●	●				
- LightSensor		●	●				
- MotionSensor		●	●				
- Bluetooth					●		●
SW-design og SW-implementering							
- LED		●	●				
- LightSensor		●	●				
- MotionSensor		●	●				
- Bluetooth				●	●		●
- GUI				●		●	
- System			●	●			
3D printet kabinet			●				
Modultest							
- PowerManagement		●	●				
- LED		●	●				
- LightSensor		●	●				
- MotionSensor		●	●				
- Bluetooth					●		●
- GUI				●		●	
Integrationstest	○						
Accepttest	○						
Diskussion	○						
Konklusion	○						
Fremtidigt arbejde	○						
Bilagsoversigt	○						

Tabel 2 - Tabel over arbejdsfordeling

2.1.6 Tidsplan



Figur 3 - Projektets tidsplan

På Figur 3 ses projektets endelige tidsplan. Der var flere finjusteringer undervejs i processen og derfor endte vi med en mere iterativ arbejdsproces, der betyder at vi er gået frem og tilbage mellem de forskellige faser.

2.1.7 Refleksioner og afvigelser

Projektet har fulgt en tydelig og struktureret tidsplan, hvilket har bidraget til et stabilt og effektivt arbejdsflow. Allerede i uge 37 blev projektformuleringen godkendt, og de efterfølgende faser, fra kravspecifikation og arkitektur til design, implementering og test, blev gennemført med løbende evalueringer og reviews.

3. Kravspecifikation

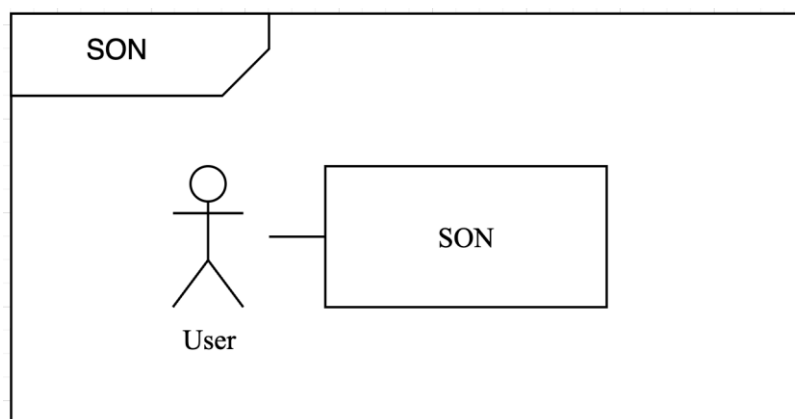
Dette afsnit fastlægger krav for projektets udførelse og danner grundlag for design, implementering og test af det samlede produkt. Formålet er at præcisere SON's funktionalitet samt de kvalitetsmæssige egenskaber, som SON skal besidde. Kravene er opdelt i funktionelle og ikke-funktionelle krav og baserer sig på systembeskrivelsen samt de udarbejdede use cases. Projektbeskrivelsen indeholder desuden ufravigelige krav, disse vil ikke blive gennemgået her, men kan findes i projektoplægget. Ved dybdegående gennemgang henvises til bilag 3.

3.1 Funktionelle krav

De funktionelle krav beskriver, hvordan SON skal fungere, og hvordan det skal reagere på brugerens interaktion. Som grundlag er der udarbejdet et aktør-kontekstdiagram samt et use case-diagram, der identificerer relevante aktører og SON's overordnede adfærd.

3.1.1 Aktør-kontekst

Til beskrivelse af SON er der opstillet et aktør-kontekstdiagram, som ses på Figur 4. Modellen er udarbejdet på et *black box*-abstraktionsniveau, hvilket betyder, at SON's interne funktionalitet ikke beskrives; der fokuseres udelukkende på forbindelserne mellem aktøren og SON's ydre grænseflader.



Figur 4 - Aktør-kontekst-diagram

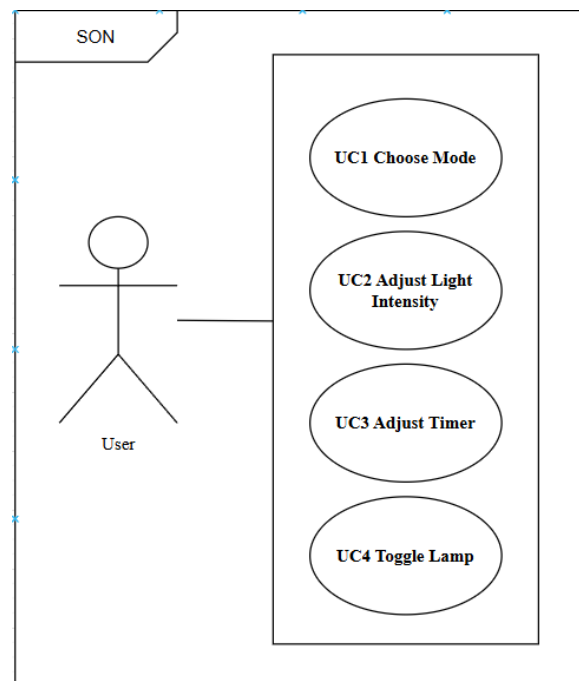
Navn:	User
Type:	Primær
Beskrivelse:	Brugeren interagerer med systemet gennem dens brugergrænseflade, hvorpå brugeren har mulighed for at vælge hvordan systemet skal operere samt se nuværende indstillinger for lampen.

Tabel 3 - Aktør-kontekst

3.1.2 Use Cases

Use case diagrammet specificerer de funktioner, som stilles til rådighed for brugeren.

Disse use cases danner grundlaget for SON's funktionelle krav.



Figur 5 - Use Case Diagram

For at gøre diagrammet mere overskueligt, er der udarbejdet en kort beskrivelse af hver UC:

UC1 Choose Mode: Brugeren har valgt at bruge SON i enten Automatic Mode eller Motion Mode, og styres enten af input fra lyssensoren, eller bevægelsessensoren.

UC2 Adjust Light Intensity: Brugeren justerer lysintensiteten af lampen.

UC3 Adjust Timer: Brugeren justerer mængden af tid, som lampen er tændt efter sidst registreret bevægelse.

UC4 Toggle Lamp: Brugeren har valgt at slukke lyset på SON, og derefter ændre mode, hvilket betyder at lampen stadig har forbindelse med GUI'en, selvom SON er slukket.

Ud fra de identificerede use cases er der udarbejdet en fully dressed use case-beskrivelse for hver. En fully dressed use case giver en detaljeret gennemgang af SON's funktionalitet i det konkrete scenarie. Nedenfor præsenteres den fulde beskrivelse for UC-1. De resterende fully dressed use cases findes i kravspecifikationen (bilag 2).

Navn:	Choose Mode
Mål:	Brugeren vælger, om SON skal styres af input fra enten bevægelsessensoren eller lyssensoren, og når lampen tændes, lyser lampen i overensstemmelse med den valgte Mode.
Initiering:	GUI'en er åben og lampen er tændt eller inaktiv.
Aktører:	Bruger
Samtidige forekomster:	1
Prækondition:	Systemet er tændt, og GUI'en er åben på hovedmenuen.
Postkondition:	Systemet er sat i valgte Mode og styres af enten bevægelsessensoren eller lyssensoren, og når lampen tændes, lyser lampen i overensstemmelse med den valgte mode.
Hovedscenarie:	1. SON's GUI viser den aktuelt valgte Mode (ved første opstart Automatic Mode). 2. Brugeren åbner underfanen for den modsatte Mode af den, systemet er i (Automatic ↔ Motion). 3. Brugeren trykker confirm i underfanen. 4. Systemet opdateres og bruger fremadrettet valgte mode. 5. GUI giver feedback i form af en indikator. 6. Når lampen tændes, lyser lampen i overensstemmelse med den valgte mode. I Automatic Mode lyser lampen, når lyset falder under en bestemt lysgrænseværdi, og i Motion Mode lyser lampen i forudbestemt tid når der registres bevægelse.
Udvidelser/Undtagelser	Use casen har ingen extensions eller undtagelser.

Tabel 4 - Fully Dressed Use Case for UC1

Den fully dressed use case for Choose Mode beskriver trin for trin, hvordan brugeren ændrer SON's driftstilstand, og hvordan systemet håndterer denne ændring. Beskrivelsen viser både brugerens handlinger og systemets efterfølgende respons. På baggrund af aktør-kontekstdiagrammet og use case-diagrammet kan de funktionelle krav identificeres. Da SON

rummer en række forskellige krav, anvendes MoSCoW-metoden til at prioritere, hvilke af disse der udgør systemets kernefunktionaliteter.

Nedenstående Tabel viser den endelige specifikation af SON's funktionelle krav. Kravene er udledt på baggrund af de opstillede use cases samt øvrige systemkrav og er prioriteret efter MoSCoW-metoden.

MoSCoW	Nr.	Krav
Must	1.	Lampen skal kunne tændes og slukkes igennem GUI'en.
	2.	Lampen skal have PWM-styring af lysintensiteten.
	3.	Lampen skal kunne operere i Automatic Mode baseret på lyssensoren.
	4.	Lampen skal kunne operere i Motion Mode baseret på bevægelsessensoren.
	5.	SON skal have en brugergrænseflade i flere niveauer, hvor lampens parametre kan ændres.
	6.	SON skal kunne lyse med maksimal intensitet i 3 timer uafhængigt af vejret.
	7.	Lampen skal være solcelledrevet og have et batteri.
Should	8.	Brugergrænsefladen bør kunne anvendes af en vilkårlig person efter 2 minutters introduktion.
	9.	SON bør have en sikker fysisk opbygning, hvor ledninger og komponenter er skjult og beskyttet.
Could	10.	Systemet kan kommunikere med GUI'en via Bluetooth.
Won't	11.	SON vil ikke have avanceret vejr- eller naturbeskyttelse.
	12.	Systemet vil ikke behandle eller opbevare data om brugeradfærd eller historiske sensorværdier.

Tabel 5 - Prioritering af funktionelle krav

3.2 Ikke-Funktionelle krav

Dette afsnit beskriver de ikke-funktionelle krav til SON. Kravene omhandler SON's kvalitetsmæssige egenskaber og er struktureret efter FURPS-modellen: *Functionality*, *Usability*, *Reliability*, *Performance* og *Supportability*. Denne inddeling giver et samlet overblik over SON's forventede kvalitetsniveau og understøtter den efterfølgende test og validering.

Nedenstående Tabel viser de ikke-funktionelle krav, som er afledt af SON's design, funktionalitet og brugergrænseflade.

FURPS	Nr	Krav
Functionality	2	Systemet skal huske indstillingerne efter GUI'en lukkes ned, og derefter starte op med samme indstillinger
Usability	6	GUI'en skal være intuitiv og nem at anvende efter kort introduktion.
Reliability	3	SON skal kunne lyse ved maksimal intensitet i minimum 3 timer på en fuld opladning.
	5	Systemet skal automatisk vende tilbage til senest anvendte tilstand efter strømafbrydelse (mode, intensitet og Timer).
Performance	1	Justering af lysintensitet skal have en forsinkelse på under 1 sekund.
Supportability / Safety	4	GUI'en må ikke tillade ændringer, der kan beskadige hardware (f.eks. for høj strøm).

Tabel 6 - Tabel over ikke-funktionelle krav prioriteret med MoSCoW

Gennem dette afsnit er der blevet opstillet en samlet kravspecifikation, som danner grundlaget for at afgrænse og tydeliggøre SON's funktionalitet. Kravene anvendes løbende i resten af rapporten som reference for design, implementering og test. I projektets afsluttende fase udføres en accepttest, der tager udgangspunkt i de opstillede funktionelle og ikke-funktionelle krav. Den tilhørende accepttestspecifikation kan findes i Kravspecifikationen og Accepttestspecifikation (bilag 2).

4. Teknisk Analyse

SON består af en række logiske blokke, som beskrevet i systembeskrivelsen afsnit 1.2.

At finde den konkrete fysiske komponent, der opfylder disse krav, er formålet med den tekniske analyse. Da projektet er et 'proof of concept', er fokus ikke på et industrielt færdigt produkt, men på dokumentation, sporbarhed og gennemsigtighed, så SON's design kan forklares, forsvares og videreudvikles. Afsnittet vil ikke indeholde den fulde tekniske analyse, men et resume over de valgte dele for hver komponent. For mere detalje refereres til Teknisk Analyse (bilag 4)

Overblikket over de analytiske valg vises i nedenstående Tabel. Tabellen viser for hver logisk funktion den valgte komponent, hvilke alternativer der er overvejet, samt hovedargumenterne fra den tekniske analyse, der ligger til grund for valget.

Komponent	Valgte løsning	Overvejede alternativer	Analyse
Mikrocontroller	Arduino Mega 2560	Arduino Nano	Flere I/O porte, tilgængelig, strømforbrug 50-100mA, kompatibel med 5V spændingsregulator
Lampe	Kitronik ZIP Circle 16 RGB LED	Enkeltstående LED'er	Individuel PWM, indbygget driver, høj fleksibilitet og energieffektivt
Solcelle	10 W solcelle panel	5 W solcelle panel	Tilstrækkeligt til 3 timers maksimaldrift, selv på vinterdage.
Solcellecontroller	10A MPPT Solcelle Controller	Eget kredsløb	Beskytter mod dybdeafledning, kompatibel med solcelle og batteri.
Spændingsregulator	LM2596 Step-Down Buck 5V 3A	LM7805 lineær regulator	Høj virkningsgrad(85%-90%), leverer stabil 5V til kredsløbets strømkrav.
Batteri	12 V Bilbatteri	Lithium-ion batteri	Tilgængeligt og kompatibelt med solcellecontrolleren.
Lyssensor	TSL2591	Fotomodstand (LDR)	Præcise lux-målinger, lettere kalibrering og lav strømforbrug (0,4mA).
Bevægelsessensor	PIR-sensor HC-SR501	Ultrasonisk sensor	Lavt strømforbrug (<1mA), god rækkevidde og let tilslutning.
GUI	C++ GUI i Qt-framework	Python GUI	Nemt at kommunikere med Arduino i C++ og reducerer middleware.
GUI kommunikation	Bluetooth HC-05	Wi-Fi modul ESP8266	Taler simpel UART, lavt strømforbrug, og stabil forbindelse.

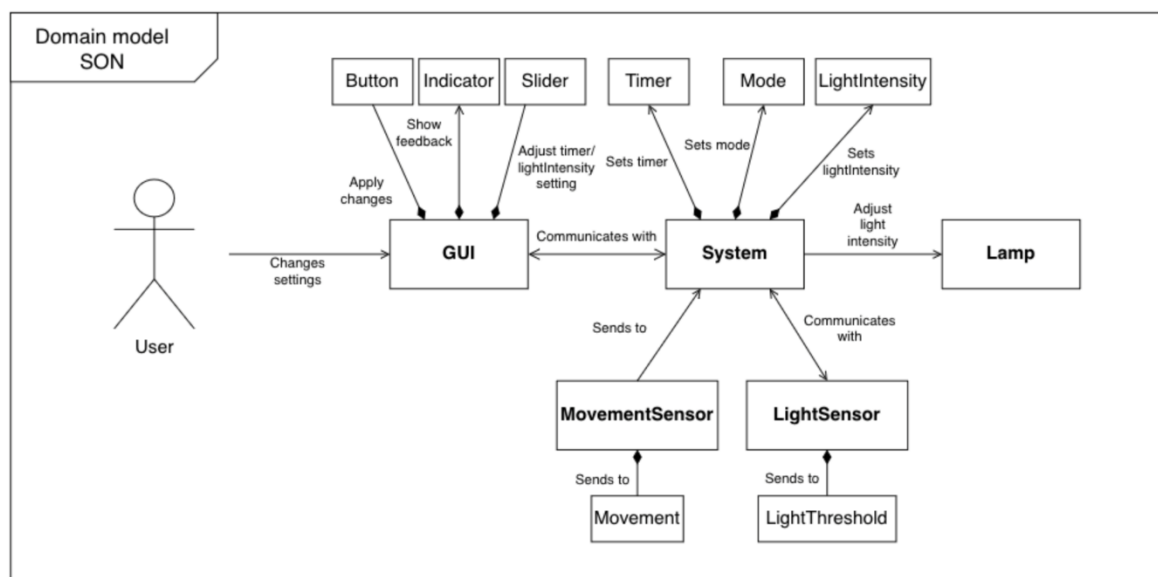
Tabel 7 - Teknisk Analyse Tabeloversigt

5. Systemarkitektur

Dette kapitel indeholder arkitekturen for SON. Arkitekturen opdeles i hardware- og softwarearkitektur, og har til formål at skabe et samlet overblik over alle systemdele samt forklare interaktionen mellem de forskellige komponenter. Den fulde systemarkitektur kan findes i bilag 5.

Systemarkitekturen er en overordnet beskrivelse og visualisering af SON's struktur. Den giver et samlet billede af de funktionelle sammenhænge og er med til at understøtte forståelsen af SON's opbygning og drift. Først præsenteres den generelle arkitektur for SON, hvorefter kapitlet opdeles i hardware- og softwarearkitektur for at beskrive henholdsvis de fysiske komponenter og den logiske styring.

For at beskrive SON er der udarbejdet en domænemodel, som illustrerer centrale begreber, deres relationer og delsystemers adfærd. Modellen afspejler SON's brug i praksis, herunder interaktionen mellem bruger, GUI, sensorer og den interne systemlogik. Domænemodellen tager udgangspunkt i fundne navneord fra use cases fundet igennem en navneordsanalyse, se bilag 6.



Figur 6 - Domænemodel for SON

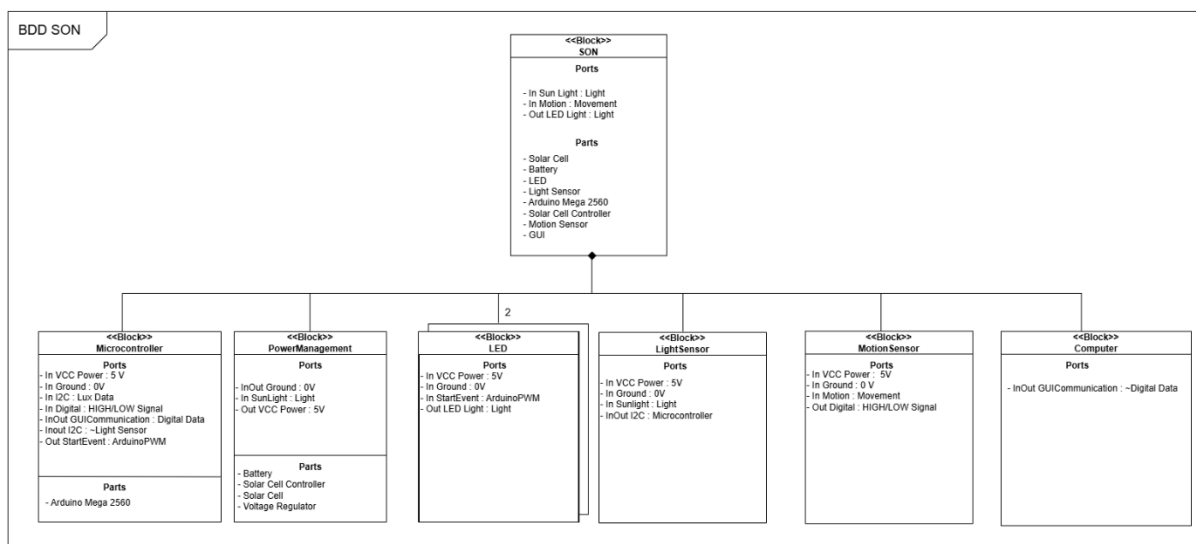
Modellen giver en klar visualisering over, hvordan de enkelte komponenter i SON fungerer og kommunikerer med hinanden.

5.1 Hardwarearkitektur

Hardwarearkitekturen beskriver den fysiske opbygning af solcellelampen og fungerer som det tekniske grundlag for design- og implementeringsfasen. Afsnittet giver et detaljeret overblik over de komponenter, der indgår i SON, samt hvordan disse er forbundet og interagerer med hinanden. Der opstilles visuelle repræsentationer af SON's struktur i form af et BDD (Block Definition Diagram) og et IBD (Internal Block Diagram). Disse diagrammer understøtter forståelsen af de overvejelser og designvalg, der ligger bag arkitekturen for den anvendte hardware.

I hardwaredelen indgår følgende centrale komponenter i SON: mikrocontroller (Arduino Mega 2560), lyssensor, bevægelsessensor, LED, solcellepanel, batteri, solcellecontroller og spændingsregulator. Disse komponenter udgør tilsammen det fysiske lag, som muliggør energiopsamling, databehandling og aktivering af lampen.

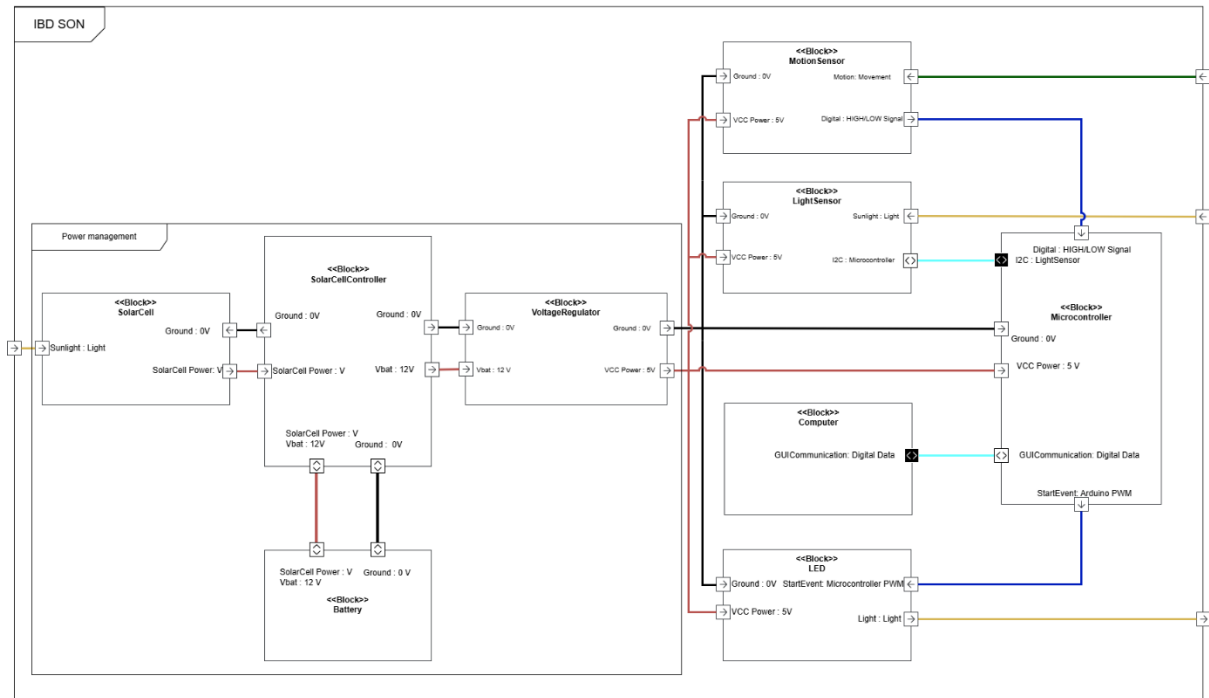
Komponenterne danner tilsammen grundlaget for SON's BDD (Block Definition Diagram), der illustrerer relationerne mellem hardwareblokkene og deres rolle i det samlede system. Projektets BDD fremgår af nedenstående figur.



Figur 7 - BDD for SON

For at supplere BDD'ets overblik over SON's hardwarekomponenter uddybes deres indbyrdes forbindelser i et IBD (Internal Block Diagram), som nedenfor illustrerer de

konkrete forbindelser, signalstrømme og interaktioner mellem komponenterne i den fysiske implementering.



Figur 8 - IBD for SON

For at give et mere detaljeret indblik i SON's interne opbygning suppleres IBD'et af en blok- og signalbeskrivelse. Beskrivelsen specificerer funktionerne for hver hardwareblok samt de signaler, der udveksles mellem komponenterne. Den samlede blok- og signalbeskrivelse fremgår i bilag 5 afsnit 3.3. Nedenfor ses kun blokbeskrivelsen.

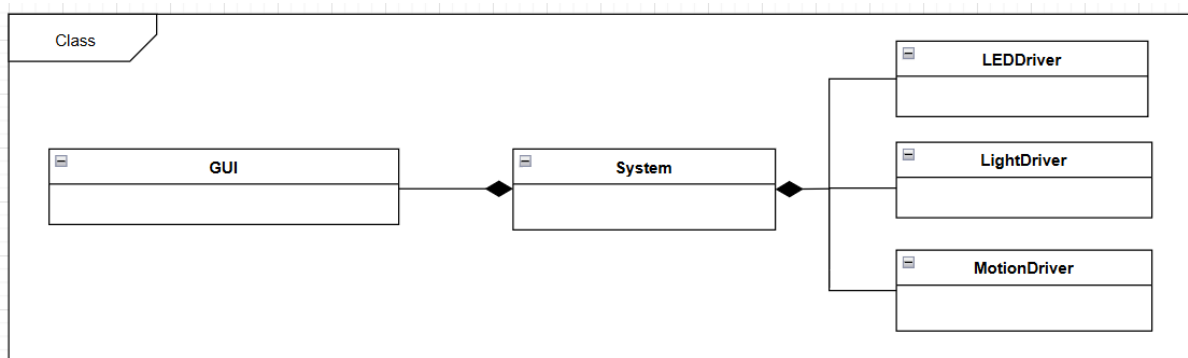
Blok navn	Funktionsbeskrivelse
SON	Overordnet styringsenhed, der koordinerer dets delkomponenter og håndterer data mellem sensorer, energistyring og output
Solarcell	Omdanner solenergi til elektrisk energi, der bruges til at oplade batteriet og forsyne systemet.
Solarcellcontroller	Regulerer opladningen af batteriet fra solcellen og sikrer korrekt ladespænding. Beskytter mod over- og afladning.
Microcontroller	Systemets centrale mikrocontroller, der styrer alle funktioner. Behandler sensorinput og styrer lysintensitet via PWM. Kommunikerer med GUI.
Battery	Lagrer energi fra solcellen.
LED	Producerer lys ved aktivering fra Arduino. Lysstyrke kan styres via PWM-signal.
Lightsensor	Måler omgivende lysintensitet for at afgøre, hvornår lampen skal tændes/sættes inaktiv i Automatic Mode.
Motionsensor	Registrerer bevægelse omkring lampen og aktiverer lyset, hvis systemet opererer i Motion Mode.
Voltageregulator	Stabiliserer spændingen fra batteri og solcellecontroller, så Arduino og sensorer modtager 5 V.

Tabel 8 - Blokbeskrivelse for BDD

Samlet set danner hardwarearkitekturen det fysiske fundament for SON, og med denne struktur på plads kan fokus nu rettes mod den softwaremæssige styring, som binder komponenterne sammen og realiserer SON's funktionalitet.

5.2 Softwarearkitektur

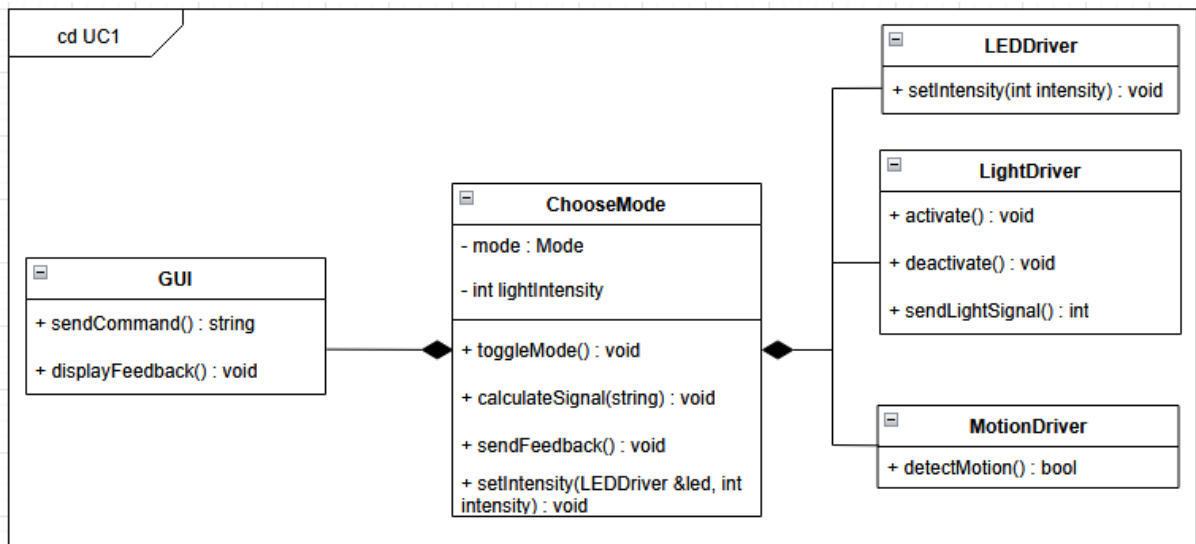
I dette afsnit præsenteres softwarearkitekturen for SON. Arkitekturen tager udgangspunkt i den udarbejdede domænemodel, som beskriver de centrale begreber, aktører og relationer. På baggrund af domænemodellen er der konstrueret et overordnet klassediagram, der illustrerer SON's struktur på et højt abstraktionsniveau, uden implementeringsdetaljer eller funktionskald. Dette diagram, som ses nedenfor, danner rammen for den efterfølgende uddybning af SON's dynamiske adfærd.



Figur 9 - Overordnet klassediagram

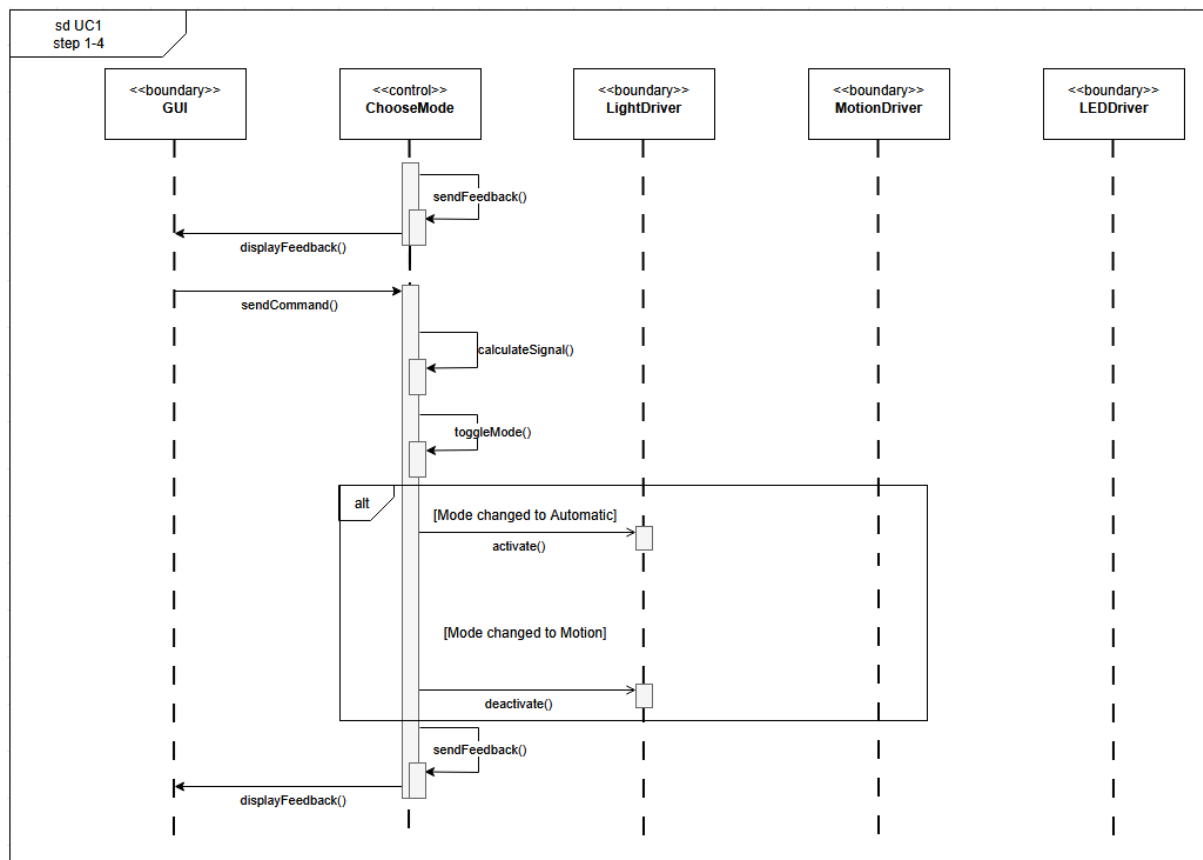
For at beskrive samspillet mellem Bruger, GUI, systemet og drivere er der desuden udarbejdet applikationsmodeller for hver af de fire use cases. Disse modeller inkluderer både klassediagrammer og sekvensdiagrammer, som tilsammen viser, hvordan systemet reagerer på forskellige typer input fra GUI og hvordan data og styringssignaler bevæger sig gennem softwaren. I dette afsnit uddybes særligt applikationsmodellen for *UC1 - Choose Mode* som eksempel på SON's funktionelle logik og interne samarbejde mellem komponenterne.

Klassediagrammet og sekvensdiagrammet er vist nedenunder:

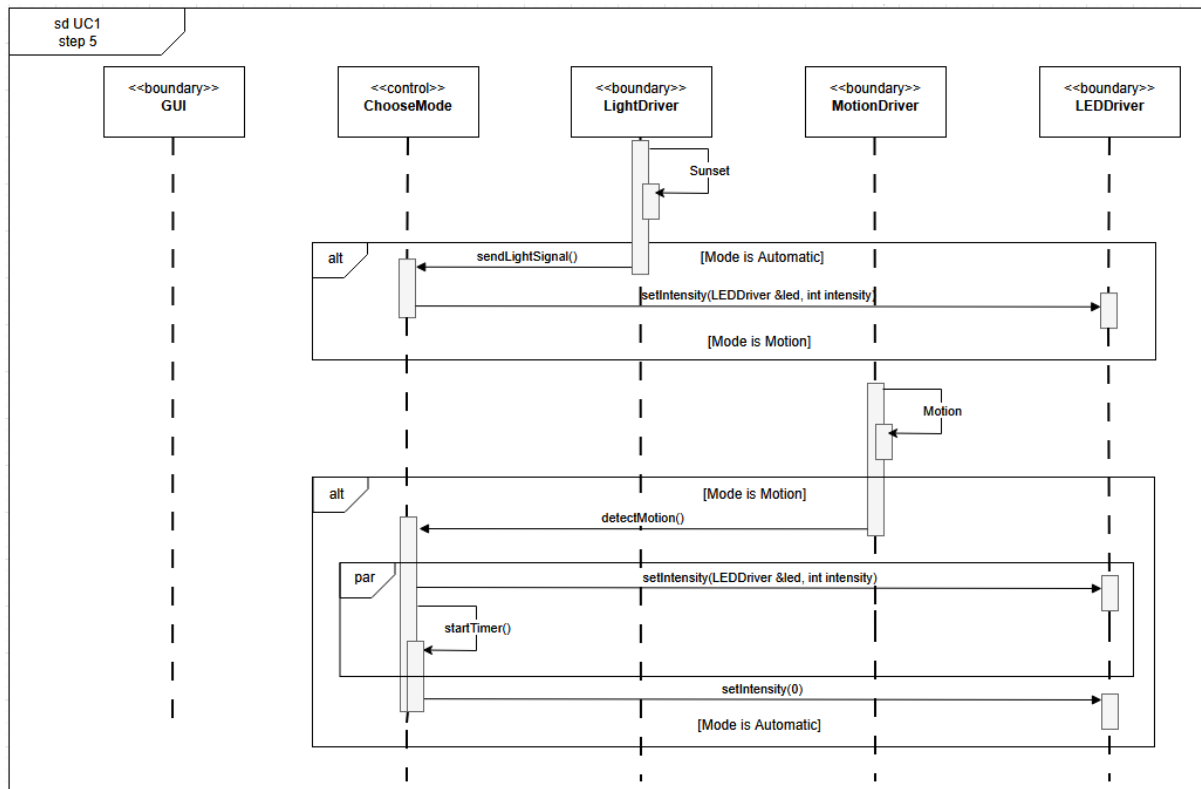


Figur 10 - Klassediagram for UC1

Sekvensdiagrammerne nedenfor illustrerer, hvordan GUI'en kommunikerer med ChooseMode-klassen, som håndterer logikken for opdatering af Mode og interaktionen med de tilhørende drivere.



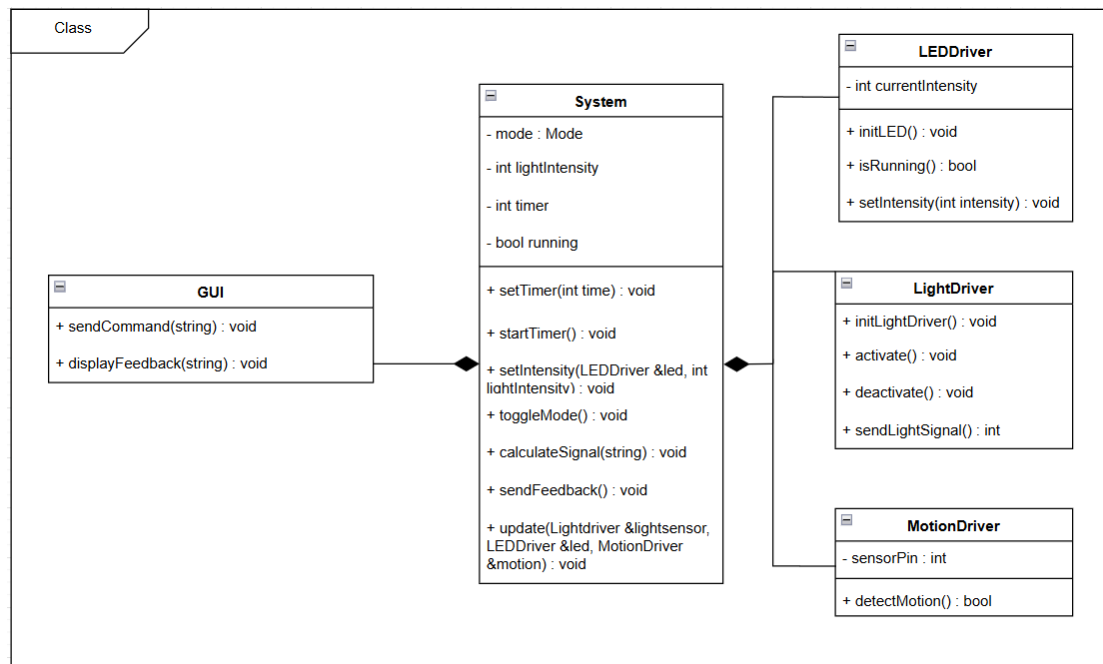
Figur 11 - Sekvensdiagram for step 1-4 i UC1



Figur 12 - Sekvensdiagram for step 5 i UC1

5.2.1 Samlet klassediagram med funktioner

Som resultat af de færdigudarbejdede applikationsmodeller, er der udarbejdet et overordnet klassediagram, som ses nedenfor.



Figur 13 - Overordnet klassediagram med funktioner

Klasserne indbefatter hver især funktioner for de opgaver, hver komponent skal kunne løse.

- **GUI:** har til opgave at give et interface til brugeren, hvorigennem der kan ses systemets nuværende indstillinger, ændres i indstillinger og gives feedback på disse ændringerne. Yderligere kommunikerer GUI med systemklassen.
- **System:** har til opgave at foretage alle logiske operationer i SON. Dette indebærer indstillinger af Timer, lysintensitet og Mode, så lampen opererer som ønsket. Disse ændringer sker ud fra de inputs brugeren giver gennem GUI. System kommunikerer ligeledes med LEDDriver, LightDriver og MotionDriver.
- **LEDDriver:** Ændrer lysintensitet baseret på input fra systemet.
- **LightDriver:** Kommunikerer med systemet om ændringer af lys i omverdenen.
- **MotionDriver:** Kommunikerer med systemet om registrering af bevægelse i omverdenen.

De enkelte funktionskald i hver klasse bliver beskrevet i Design og Implementerings dokumentet (bilag 7)

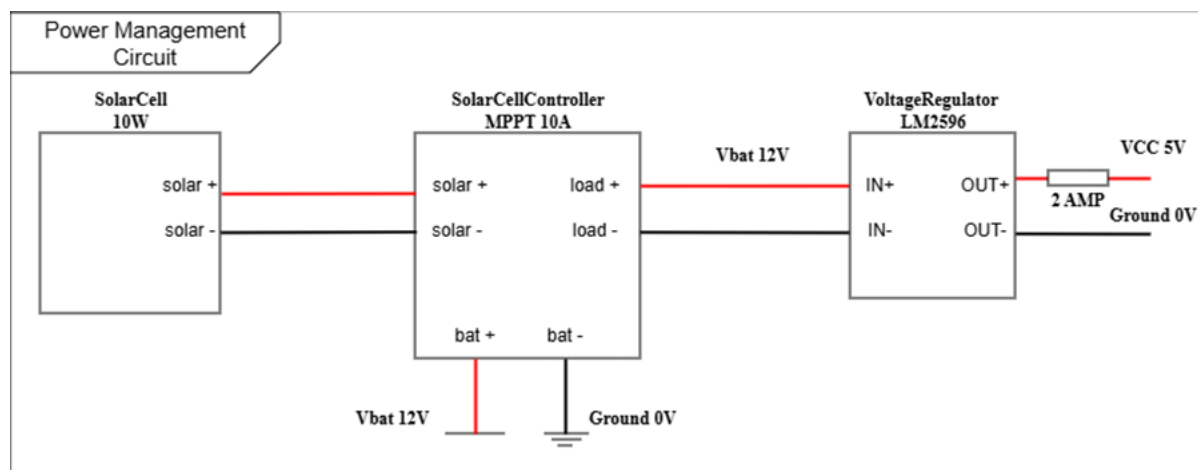
6. Hardware Design og Implementering

Dette afsnit dykker dybere ned i hardware designet, der er essentielt for den solcelledrevne lampes funktionalitet kan leve op til kravene specificeret i kravspecifikationen. Hovedfokus i designfasen var den effektive integration af de centrale komponenter for at skabe et energieffektivt system. Ved dybdegående gennemgang henvises til bilag 7.

6.1 PowerManagement

Designet af PowerManagement var essentielt for at etablere den nødvendige strømforsyningsinfrastruktur til den solcelledrevne lampe. PowerManagement er konstrueret som en selvstændig hardwarekomponent, hvis primære formål er at opsamle, lagre og stabilt distribuere energi til hele systemet, med undtagelse af GUI'en, som forsynes eksternt fra brugerens enhed.

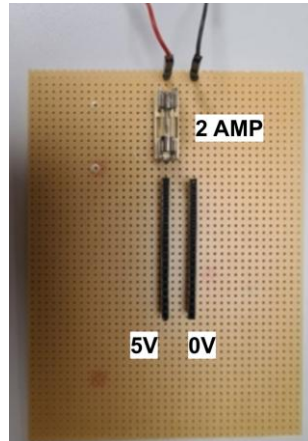
Kernestyringen varetages af en MPPT Solcellecontroller (10A), som håndterer opladningen af 12V bilbatteriet og beskytter mod dybdeafledning. Valget af en 10W solcelle sikrer, at PowerManagement kan generere nok energi til at opretholde de specificerede driftstider. De elektriske forbindelser er illustreret i kredsløbsdiagrammet nedenfor.



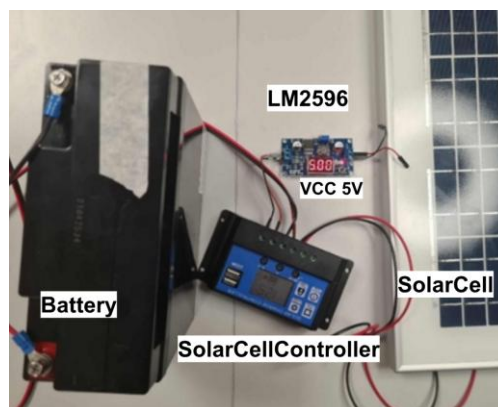
Figur 14 - PowerManagement-kredsløb

Den lagrede energi konverteres til en stabil 5V udgangsspænding via en Step-Down spændingsregulator (LM2596), der forsyner mikrokontrolleren, sensorer og LED'er. For at beskytte elektronikken mod overstrøm er der indsat en 2A sikring. Størrelsen af denne sikring baserer sig på det teoretiske maksimale strømtræk, som er beregnet i Teknisk Analyse (bilag 4)

Den fysiske implementering, herunder fordelingen af 5V via veroboard og kabelføringen, kan ses i de to nedenstående figurer.



Figur 15 - Implementering af 5V fordeler



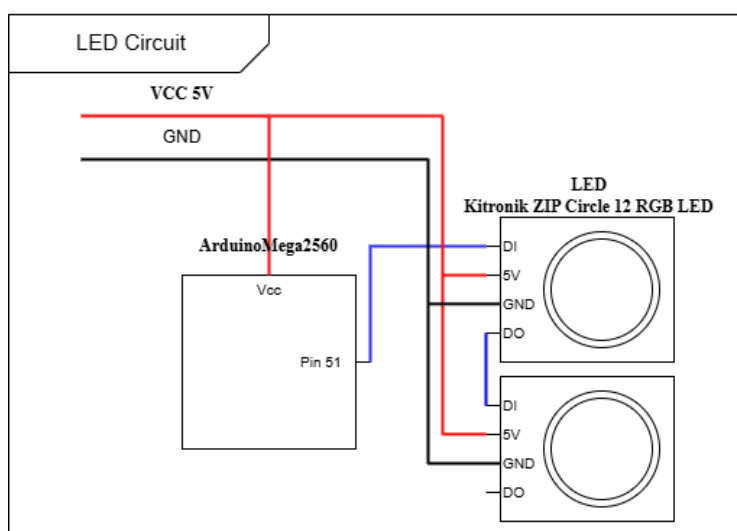
Figur 16 - Implementering af PowerManagemen

6.2 LED

Designet af selve belysningskilden fokuserer på at levere en effektiv og fleksibel lyskilde, der understøtter alle SON's driftsmodes. Der er valgt to Kitronik ZIP Circle (16 RGB ZIP LED's) for at sikre den nødvendige lysstyrke og dækning. Hver af de 16 LED'er kan styres individuelt, hvilket giver mulighed for præcis farve- og intensitetsregulering.

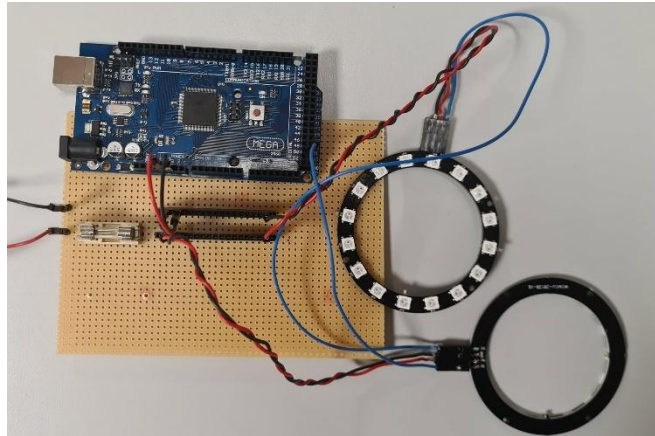
LED-kredsløbet er designet til at opfylde de centrale krav om at kunne tændes/slukkes via GUI'en og have PWM-styring af lysintensiteten. Dette opnås ved at forbinde LED'ernes datalinje (DI) til en digital pin (pin 51) på Arduino Mega 2560. Mikrokontrolleren håndterer al logikken for PWM-regulering og aktivering i SON's forskellige driftsmodes (Automatic Mode og Motion Mode).

De elektriske forbindelser, der sikrer stabil forsyning af 5V fra PowerManagement, er detaljeret i kredsløbsdiagrammet nedenfor.



Figur 17 - Kredsløb til LED

Strømforsyningen leveres fra PowerManagement-kredsløbet, og de to LED-cirkler benytter et fælles datainput. Den fysiske implementering, hvor der er lagt vægt på at minimere støj, ved at snurre ground omkring VCC og sikre robusthed med jumperkabler, kan ses her i nedenstående figur.



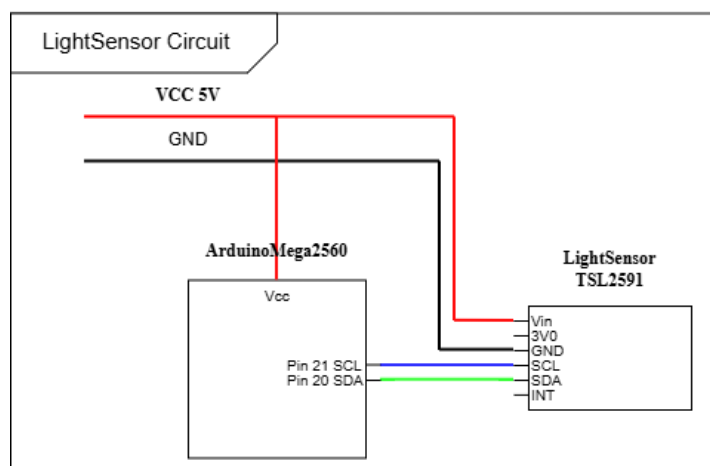
Figur 18 - Implementering af LED

6.3 Lightsensor

Lyssensoren udgør en central del af SON's automatisk styring, da den registrerer omgivelsernes lysniveau og dermed afgør, hvornår lampen skal aktiveres i Automatic Mode.

SON anvender TSL2591-lyssensoren, der kommunikerer med Arduinoen via I2C-protokollen. Sensoren kan måle et bredt spektrum af lysintensiteter, hvilket gør den velegnet til udendørs brug, hvor både sollys og svag belysning skal kunne detekteres pålideligt. Sensoren forsynes med en stabil 5V fra spændingsregulatoren, og fælles stel sikrer konsistent reference mellem sensor og mikrocontroller.

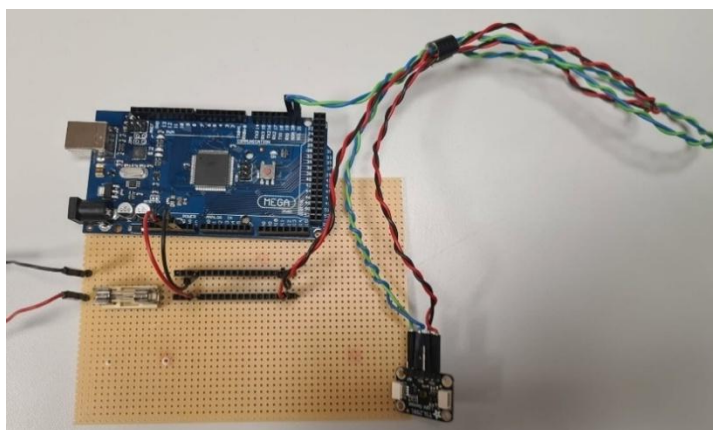
SDA- og SCL-linjerne er snurret omkring hinanden for at sikre stabil I2C-kommunikation, og forbindelserne benytter jumperkabler.



Figur 19 - Kredsløbsdiagram til lyssensor

I kredsløbsdiagrammet ovenfor vises tilslutningen mellem lyssensor, Arduino og spændingsregulatoren, samt signalflowet over I2C-bussen.

Den fysiske implementering, inklusive sensormontering og kabeltræk, fremgår af figuren nedenfor.



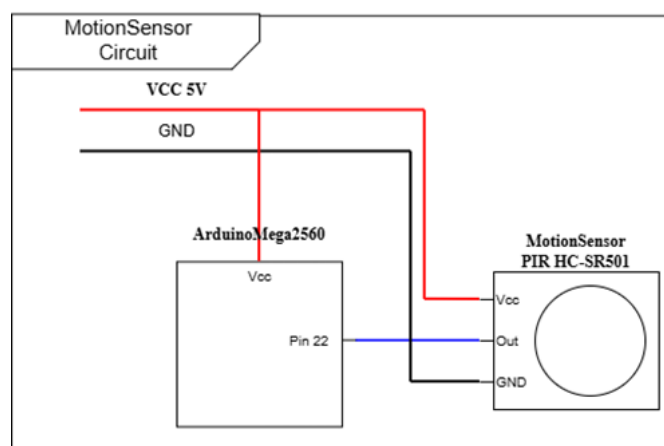
Figur 20 - Implementeret lyssensor kredsløb

6.4 MotionSensor

Bevægelsessensoren er en afgørende komponent i SON's Motion Mode og gør det muligt at aktivere lampen, når der registreres bevægelse i omgivelserne.

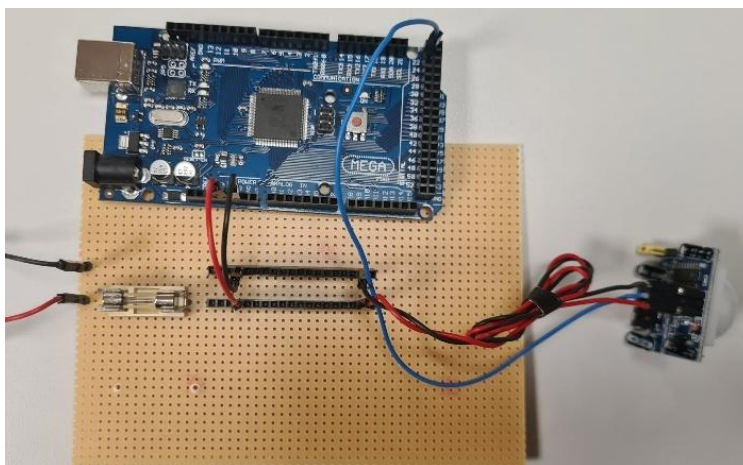
PIR-sensoren leverer et digitalt HIGH/LOW-signal til Arduinoen, hvilket gør det enkelt at anvende signalet i systemet. Sensorkomponenten forsynes med en stabil 5V fra spændingsregulatoren og fælles stel.

I kredsløbsdiagrammet nedenfor fremgår forbindelsen mellem PIR-sensoren, mikrocontrolleren og spændingsregulatoren, herunder det digitale outputsignal og forsyningslinjerne.



Figur 21 - Kredsløbsdiagram til bevægelsessensor

Nedenstående figur viser den fysiske implementering af MotionSensor.



Figur 22 - Implementeret bevægelsessensor kredsløb

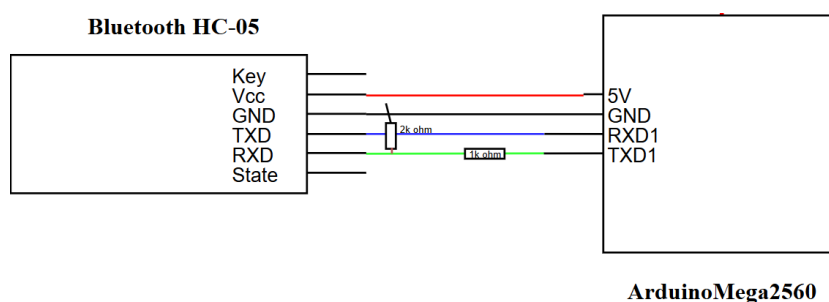
6.5 Bluetooth

Vi har valgt at bruge Bluetooth som vores kommunikation mellem GUI og systemet. Dette var tidligere et ”could” krav i vores MoSCoW fordeling. Valget af denne enhed blev truffet, da der var vurderet tilstrækkelig tid til implementering. Bluetooth-modulet udgør den centrale kommunikationsenhed mellem systemet (mikrocontroller) og brugerens GUI-applikation. Modulet muliggør trådløs udveksling af kommandoer og feedback, hvilket er afgørende for, at brugeren kan interagere med lampen uden fysisk adgang til systemets hardware.

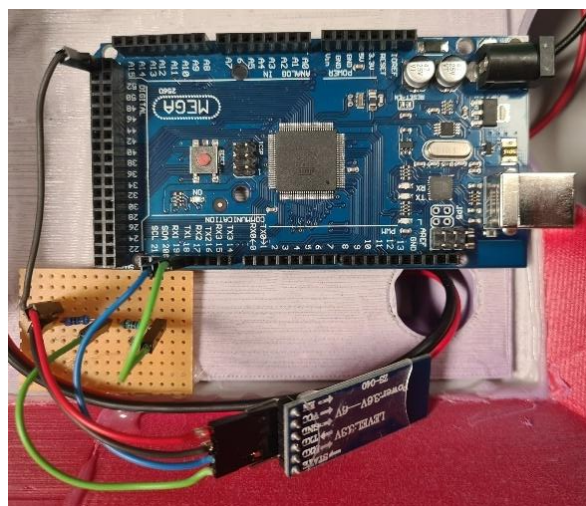
SON anvender HC-05 Bluetooth-modulet, da det understøtter Seriel Port Profil (SPP) og dermed fungerer som en virtuel UART-forbindelse mellem GUI og Arduinoen. Modulet er forbundet til Arduino Mega 2560's dedikerede serielle interface (Serial1), hvilket sikrer, at kommunikationen er isoleret fra andre interne processer og derfor kan behandles stabilt og uden konflikter.

HC-05-modulet forsyner Arduinoen med indkommende kommandoer om Mode, lysintensitet, Timer og tænd/sluk-status, og sender samtidig feedback tilbage til GUI'en om systemets aktuelle tilstand. For at sikre korrekt drift er modulet forsynet med en stabil 5 V fra spændingsregulatoren, og fælles stel etablerer et fælles referencepunkt mellem kommunikationsmodulet og mikrokontrolleren.

Modulets elektriske tilslutning, inklusive RX/TX-linjerne og forsyningsforbindelserne, fremgår af kredsløbsdiagrammet i nedenstående Figur, mens den fysiske implementering vises herefter.



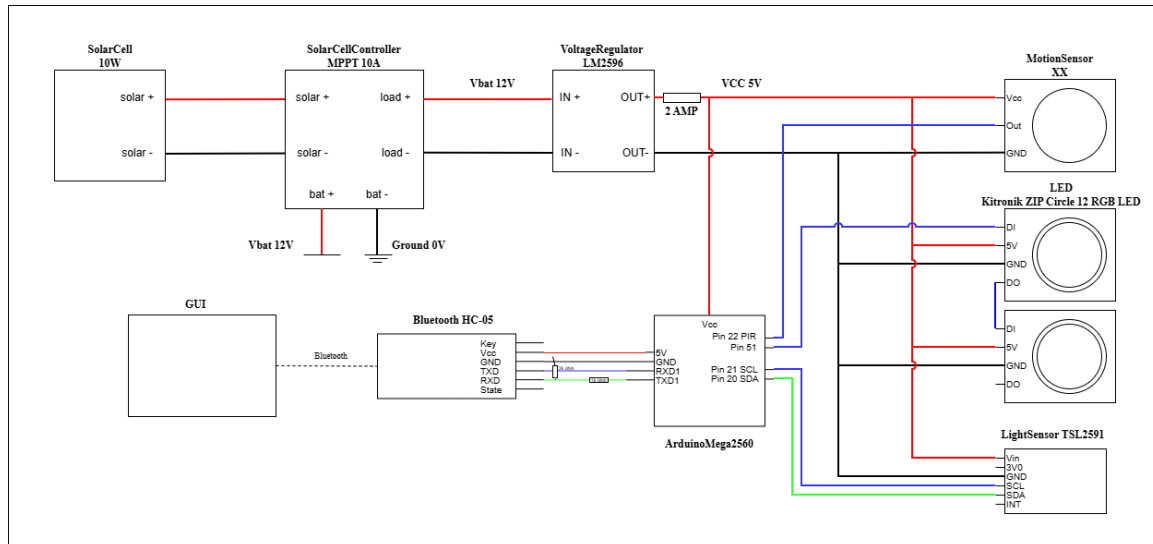
Figur 23 - Kredsløbsdiagram til Bluetooth



Figur 24 - Implementering af Bluetooth-kredsløb

6.6 Samlet hardware-overblik

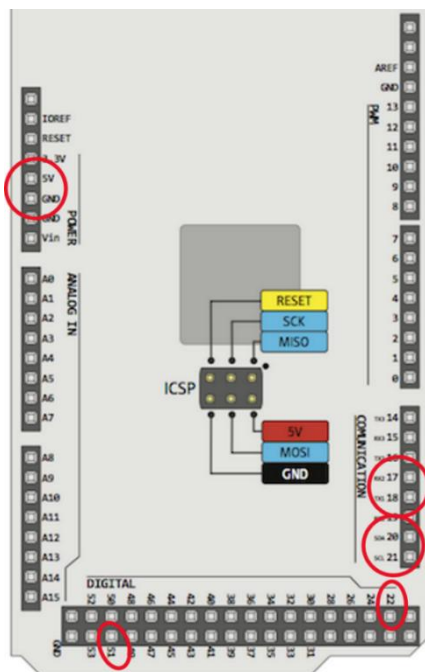
Ved at kombinere kredsløbsdiagrammerne og forbindelserne gennemgået i ovenstående afsnit er det muligt at lave et kredsløbsdiagram, der beskriver de samlede hardwaredele i SON.



Figur 25 - Kredsløbsdiagram til SON

6.6.1 Arduino forbindelser

Nedenstående er et overblik over hvordan hardwarekomponenterne er forbundet til Arduinoens forskellige pins:



Figur 26 - Billede af Arduino-forbindelserne³

6.6.2 Strømstyring og energistyring

VCC og GND fra spændingsregulatoren er direkte forbundet til 5V pin og GND på Arduinoen.

6.6.3 LED

Pin 51 er output pin fra Arduinoen, og bruges til at styre PWM-signalet til LED ringene.

6.6.4 LightSensor

Pin 20 (SDA) og pin 21 (SCL) på Arduinoen er forbundet til lyssensoren. SDA bruges til at kommunikere frem og tilbage, og SCL fortæller hvornår data skal læses eller skrives.

6.6.5 MotionSensor

Pin 22 bruges til at modtage et digitalt signal fra bevægelsessensoren. Hvor HIGH = bevægelse.

³ (Pinout, u.d.)

6.6.6 GUI

GUI kommunikerer igennem Bluetooth med modulet HC-05. Både VCC (5V) og GND er suppleret af Arduinoen.

Pin 19 (RX1) på Arduinoen er forbundet med TXD på HC-05.

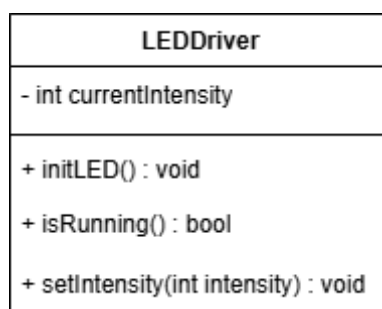
Pin 18 (TX1) på Arduinoen er forbundet med RXD på HC-05.

7. Software Design og Implementering

Nedenfor følger en designbeskrivelse af hver af SON's softwareklasser. Hver klasse introduceres med en analyse af fundne funktionaliteter, der skal realiseres. Hver enkelt afsnit er udarbejdet med fokus på de valg der ligger bag designet og uddyber de mest essentielle funktioner. Ved dybdegående gennemgang henvises til bilag 7. Softwaredesignet er udarbejdet ved hjælp af C++ i VS Code med PlatformIO som udviklingsmiljø.

7.1 LED

I nedenstående figur fremgår klassediagrammet for LEDDriver.



Figur 27 - LEDDriver klassediagram

LEDDriver-klassen er ansvarlig for styringen af SON's LED'er. Klassen indgår som en selvstændig driverkomponent i softwarearkitekturen. Driveren udvikles så den understøtter enkel udvidelse og kan udskiftes uden påvirkning af resten af arkitekturen og indeholder følgende 3 metoder:

- initLED()
- setIntensity(int)
- isRunning()

Formålet er at abstrahere al hardwarekommunikation væk fra resten af systemet, så LED-styring kan foretages gennem et enkelt interface.

Vi benytter os af de eksisterende WS2812B drivere. Den aktuelle implementering bliver beskrevet i Modultest og Integrationstest (bilag 8) afsnit 3.1.

7.1.1 Konstruktion og hardware-binding

Når LEDDriver-objektet oprettes, initialiseres en `ws2812b_handle_t` struktur. Denne håndterer al SPI- og delay-funktionalitet via funktionspointere. Denne driver er forud implementeret.

Denne opbygning adskiller hardware fra logik og muliggør, at kommunikationslaget senere kan udskiftes (f.eks. SPI → bit-banging eller I2C), uden at LEDDriver-klassen skal omskrives.

7.1.2 Initialisering - `initLED()`

Initialiseringsfunktionen klargør SPI-interfacet og nulstiller LED-ringen. Alle LED'er sættes til inaktiv tilstand ved at skrive farven `0x000000` til hele buffer-området.

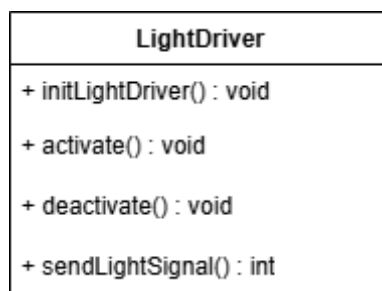
Fordelen er at systemet opnår en entydig defineret starttilstand, og at eventuelle resterende værdier fra tidligere kørsel elimineres.

7.1.3 Styring af lysintensitet - `setIntensity(int)`

Lysstyrken håndteres af `setIntensity()`, som modtager en værdi mellem 0-100 %. Værdien checkes for at undgå ugyldige input, hvorefter den mappes lineært til et 8-bit lystal (0-255). Der bruges en ensfarvet hvid kanalstyring, hvilket betyder at R, G og B sættes til samme styrkeniveau for at skabe neutralt hvidt lys.

7.2 LightDriver

I nedenstående figur fremgår klassediagrammet for LightDriver.



Figur 28 - LightDriver klassediagram

LightDriver-klassen har til formål at indhente omgivende lysdata via TSL2591-lyssensoren, som kommunikerer over I2C-bussen. Dette afsnit vil gå i dybden med I2C kommunikationen og bygger på Systemarkitektur (bilag 5) afsnit 5.1 I2C. Klassen abstraherer samtlige registertilgange og kommunikationssekvenser, så resten af systemet kan arbejde med et simpelt digitalt output: $1 = nat$ og $0 = dag$. Derved indgår LightDriver som et sensorinput til System-klassen, som anvender resultatet til automatisk lysstyring.

7.2.1 Lavniveau I2C-kommunikation

Da TSL2591-sensoren kommunikerer via I2C, implementeres protokollen direkte i driveren uden brug af eksterne biblioteker. Dette øger kontrollen og sikrer hardwareforståelse, men kræver samtidig korrekt opsætning af busdrivning, acknowledge-styring og adressehåndtering.

Driveren implementerer:

Funktion	Beskrivelse
i2c_init()	Initierer I2C, clock = 100 kHz ved 16 MHz CPU
i2c_start() / i2c_stop()	Danner start-/stop-betingelser på bussen
i2c_write(uint8_t)	Sender én byte data med ACK-ventning
i2c_read_ack() / i2c_read_nack()	Læser data med/uden ACK afhængigt af sidste byte

Tabel 9 - Funktionsbeskrivelse af I2C funktioner

7.2.2 Initialisering - initLightDriver ()

Initieringsmetoden udfører følgende trin:

1. Klargør I2C-bussen

2. Læser ID-registret (0x12) → validerer sensor tilstedeværelse
3. Sætter integrationstid til 100ms
4. Sætter gain (sensitiviteten af sensoren) til medium (0x10)

Denne sekvens sikrer, at driveren starter i en kendt og stabil målekonfiguration.

7.2.3 Sensoraktivering - activate() og deactivate()

Lyssensorens strømforbrug styres aktivt gennem activate() og deactivate(). Sensoren tændes med ALS-måling aktiveret og kan efterfølgende aflæse intensitet kontinuerligt. Dette muliggør energibesparelse ved periodisk aflæsning.

7.2.4 Lyismåling og hysteresis - sendLightSignal()

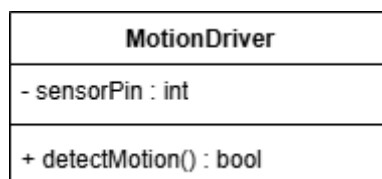
Sensoren aflæser CH0-kanalen (full-spectrum lys) via to 8-bit registre, som flettes til 16-bit opløsning. Resultatet indgår efterfølgende i en hysteresisimplementering for at undgå hurtige skift mellem mørk/lys ved grænseværdier. Systemet anvender faste threshold-værdier.

Hysteresis-designet forhindrer flimrer ved grænseområdet og leverer stabil binær udgangsværdi. Det betyder, at System-klassen kan anvende lyssensoren som en robust digital trigger fremfor rå måledata.

LightDriver håndterer al lavniveau relateret I2C-kommunikation og registeradgang til TSL2591-lyssensoren. Modulet tilbyder et klart og minimalt interface bestående af initiering, aktivering/deaktivering samt digital lysstatus.

7.3 MotionDriver

I nedenstående figur fremgår klassesdiagrammet for MotionDriver.



Figur 29 - MotionDriver klassesdiagram

MotionDriver udgør systemets PIR-bevægelsessensor og leverer et simpelt digitalt output, som angiver hvorvidt der registreres bevægelse i systemets omgivelser.

Driveren er designet med fokus på minimal konfiguration og direkte hardwareadgang via AVR-registret PINA. Da PIR-modulet opererer digitalt kan interfacet holdes markant simplere end LightDriver og LEDDriver.

7.3.1 Pin-initialisering

Klassen modtager et indgangsben som parameter og understøtter i nuværende implementation PIR-sensor monteret på pin 22, som mappes til PA0.

Ved konstruktion sættes pin-konfigurationen direkte i data direction registeret (DDRA). Driveren tillader dermed nem hardwareudskiftning ved at tilføje flere case-grene for alternative pins.

7.3.2 Bevægelses-detektion - detectMotion()

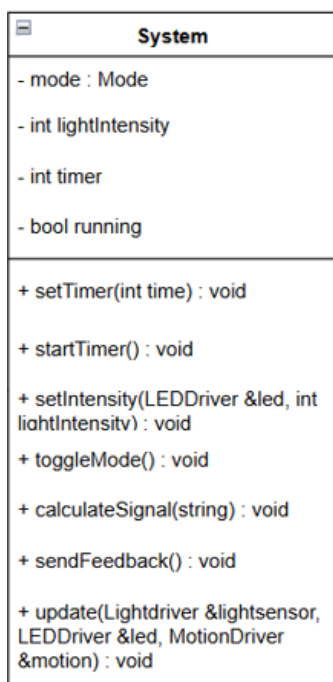
Registrering af bevægelse udføres ved et enkelt digitalt read på inputregisteret PINA, hvilket gør metoden både hurtig og strømbesparende. PIR-moduler leverer typisk logisk HIGH ved aktivitet, hvilket afspejles direkte i returnværdien.

Metoden returnerer true, når sensoren registrerer bevægelse, og false ellers. Da PIR-output er relativt langsomt og har naturlig hysteresi i sensorhardwaret, kræver systemet ingen yderligere software.

7.4 System

I dette afsnit beskrives designet af systemklassen, som binder alle softwaremoduler sammen og sikrer, at SON opfører sig i overensstemmelse med både kravspecifikation og systemarkitektur.

I nedenstående figur er systemklassen opstillet som UML-diagram.



Figur 30 - Systemkode UML- diagram

7.4.2 Constructor og initialisering

Ved første opstart skal systemet sættes i en veldefineret grundtilstand. Ud fra STM-diagrammet i Systemarkitektur (bilag 5) og de valgfrie projektbeslutninger initialiseres systemet som følger:

- Systemet starter i **Automatic Mode**
- Lampen starter i **tændt tilstand**
- Første lysintensitet sættes til **50 %**
- Systemets gemte Timer værdi sættes til **10 sekunder**

Constructoren sætter disse værdier, så systemet altid har en stabil, forudsigelig opførsel før første brugerinput.

7.4.3 Setters og basale tilstandsfunktioner

Systemet indeholder flere simple setter-funktioner, som blot opdaterer interne attributter:

- **setTimer()**
- **toggleMode()**
- **setIntensity()**
 - Denne er udvidet: udover at opdatere en attribut kaldes også `LEDDriver::setIntensity()`, så den fysiske lysstyrke også ændres.

7.4.4 Ikke-blokerende Timer (`startTimer`)

I Motion Mode skal lampen forblive tændt i et givent tidsrum efter en bevægelse. Af hensyn til kravspecifikationen (maks. 1 sekund forsinkelse for GUI-input) må systemet ikke anvende et blokerende delay.

Derfor benyttes en Interrupt Service Routine (ISR), drevet af en af AVR-mikrokontrollerens timere.

ISR'en aktiveres, når timeren når sin compare-værdi, hvorefter en tællervariabel nedtælles. Dette gør det muligt for CPU'en at fortsætte med at behandle input fra GUI'en, selv mens timeren kører i baggrunden.

Timerens opløsning er 10 ms, og derfor sættes $counter = s \cdot 100$ for et delay på s sekunder. Når counteren når 0, returnerer funktionen **isDone() = true**, hvilket fortæller systemet at timeren er udløbet. Dette delay implementeres i sin egen header og cpp fil.

7.4.5 Bluetooth-håndtering: `calculateSignal()` og `sendFeedback()`

Herudover i systemkoden, defineres `calculateSignal()`, som modtager et byte-array fra Bluetooth, samt `sendFeedback()`, som sender systemets nuværende status (attributter) tilbage til GUI gennem Bluetooth. Der bruges en switch case i `calculateSignal()`, da GUI sender et startbogstav (A-E), som aflæses, og derefter bruges værdien efter bogstavet i switch casen. En simpel switch case kunne se ud som på nedenstående figur.

```
SWITCH byte-array  
CASE "A"  
toggleMode(value)  
BREAK  
CASE "B"  
setIntensity(value)
```

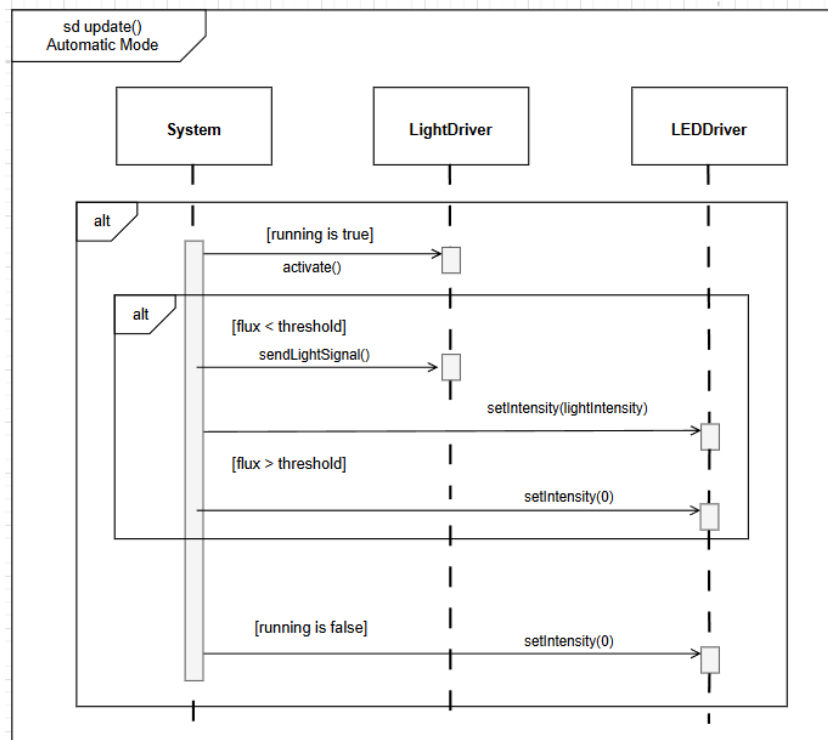
Figur 31 - Simpel Switch-case

Switch-case-strukturen kan efterfølgende udvides med de resterende kommandoer og de tilhørende definitioner beskrevet i afsnit 5.2 i Systemarkitekturen (Bilag 5).

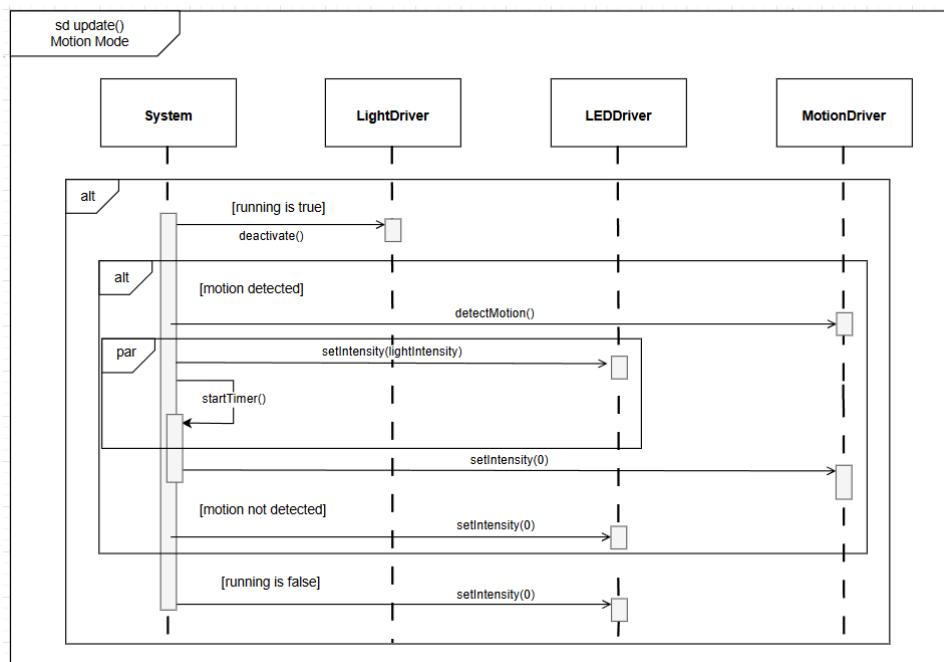
Funktionen **sendFeedback()** indebærer ingen yderligere kompleks logik; den samler systemklassens aktuelle attributværdier i en samlet buffer, som efterfølgende transmitteres via UART.

Funktionen `update()` er systemets centrale loop. I henhold til de ikke-funktionelle krav (krav 5.2, bilag 2) skal lampen slukkes, når brugeren deaktiverer den via GUI'en, men systemets konfigurationsparametre (fx `lightIntensity`) skal fortsat bevares. Dette håndteres ved hjælp af attributten `bool running`, som angiver, om lampen senest blev aktiveret eller deaktiveret af brugeren.

Hvis `running` er `true`, udføres systemets logik som specificeret i nedenstående sekvensdiagrammer:



Figur 32 - update() - Automatic Mode



Figur 33 - update() - Motion Mode

Figurene ovenfor illustrerer, hvordan systemet internt behandler sensorinput og styrer de tilknyttede drivere i henholdsvis Automatic Mode og Motion Mode. Diagrammerne viser det overordnede loop, som får SON til at fungere i praksis og sikrer, at systemet følger både arkitektur og kravspecifikation.

Selvom alle funktionelle dele af systemet nu er beskrevet, mangler der stadig at blive adresseret et af de ikke-funktionelle krav:

Krav 5: Efter en strømafbrydelse eller systemfejl skal lampen automatisk vende tilbage til sin senest anvendte tilstand (mode, lysintensitet m.m.) uden brugerens indblanding.

Dette håndteres ved at anvende mikrocontrollerens indbyggede EEPROM. Da EEPROM er ikke-flygtig hukommelse, kan systemets indstillinger gemmes permanent under strømsvigt. EEPROM tildeles en selvstændig header- og cpp-fil, som indeholder funktioner til at gemme og hente de relevante systemværdier. Disse funktioner benyttes bl.a. når der modtages nye konfigurationsdata fra GUI'en.

Derudover implementeres en separat UART-fil, som håndterer Bluetooth-kommunikation og opsætning. Dette beskrives nærmere i afsnit 7.5.

7.4.6 Main kode

Main-koden holdes bevidst minimal for at følge god programmeringspraksis. Initieringen startes med at køre alle inits(), sætte pins og oprette de nødvendige objekter. Herefter køres while(1) loopet. I loopet tjekkes der for beskeder fra GUI, og hvis der modtages en besked, køres calculateSignal(). Hvis der ikke modtages en besked, køres update() loopet.

7.5 Bluetooth

UART-kommunikationen mellem Arduino Mega 2560 og HC-05 Bluetooth-modulet implementeres via mikrocontrollerens dedikerede USART1. Denne implementering er designet som et lavniveau-modul, der udelukkende håndterer datatransmission og -modtagelse, og derved abstraheres registeradgangene fra systemets øvrige logik. Nedenstående Tabel giver et overblik over Bluetooth-modulets funktionalitet.

Funktion/Variable	Beskrivelse
uartBuffer[32]	Buffer til modtagne tegn, afsluttes med \0 når linje er klar
uartIndex	Aktuel position i uartBuffer, øges ved hvert modtaget tegn
lineReady	Flag der signalerer, at en komplet linje er modtaget
uart_init()	Initialiserer USART1 med 9600 baud, 8N1, aktiverer RX/TX og RX interrupt
uart_send_char(char)	Sender ét tegn via UART
uart_send_string(str)	Sender en streng via UART, tegn for tegn
ISR(USART1_RX_vect)	Modtager tegn, gemmer i buffer, sætter lineReady når linje afsluttes

Tabel 10 - UART-kommunikation funktioner og variable

7.5.1 Initialisering af UART - uart_init()

Funktionen `uart_init()` står for den grundlæggende opsætning af det serielle interface. UART'en konfigureres til en baudrate på 9600 bps, hvilket ved en systemfrekvens på 16 MHz opnås ved at sætte `UBRR`-registret til 103. Herefter aktiveres både sender (`TXEN1`) og modtager (`RXEN1`) i `UCSR1B`, og modtage-interruptet (`RXCIE1`) slås til, så indkommende data kan håndteres asynkront. `UCSR1C` opsætter rammen til standardformatet 8 data-bits, én stopbit og ingen paritet. Til sidst aktiveres globale interrupts, hvilket gør det muligt for modtage-interruptet at blive udløst, så snart HC-05-modulet sender data.

7.5.2 Datamodtagelse (ISR)

Al modtagelse af data foregår gennem et interrupt, der aktiveres, når et nyt byte er tilgængeligt i `UDR1`. ISR'en læser det modtagne tegn og håndterer bufferen løbende. Hvis tegnene udgør en afsluttet linje dvs. når `\n` eller `\r` registreres og bufferen indeholder indhold, afsluttes strengen automatisk med en null-terminator. `uartIndex` nulstilles, og statusflaget `lineReady` sættes til true, hvilket signalerer til hovedprogrammet, at en komplet kommando er klar til behandling i `calculateSignal()`. Hvis der stadig modtages almindelige tegn, gemmes de fortløbende i `uartBuffer`, så længe der er plads i bufferen, for at undgå overflow.

7.5.3 Datatransmission

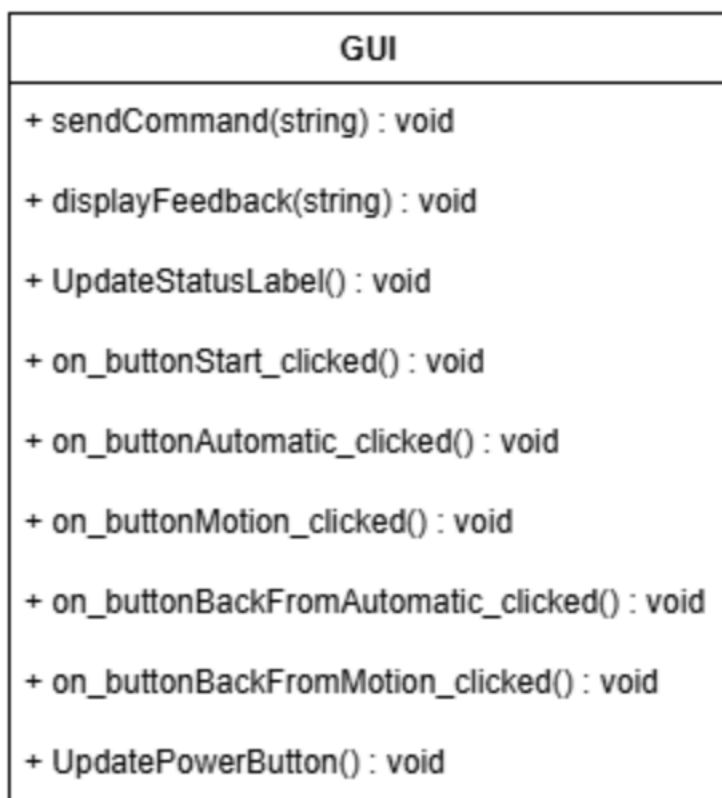
To funktioner er implementeret til transmission:

- `uart_send_char(char c)`: Sender en enkelt karakter. Funktionen anvender polling ved at vente i et while loop, indtil UDRE1-flaget i UCSR1A er sat, hvilket indikerer, at data-registret er tomt og klar til at modtage et nyt byte.
- `uart_send_string(const char* str)`: En simpel wrapper-funktion, der iterativt kalder `uart_send_char` for hvert karakter i den null-terminerede inputstreng, indtil strengens ende er nået.

Denne opbygning sikrer, at modtagelse af data sker asynkront og ikke blokerer systemets hovedprogram, hvilket er essentielt for at opretholde en dynamisk adfærd, som krævet af ikke-funktionelle krav.

7.6 GUI

I dette afsnit beskrives designet og implementeringen af SON's grafiske brugerinterface (GUI), herunder dets opbygning, funktionelle ansvar og rolle som bindeled mellem brugerens input og de kommandoer, der sendes til Arduino-enheden.



Figur 34 - GUI Klassediagram

Klassediagrammet ovenfor viser de centrale metoder i GUI-laget, som tilsammen håndterer navigation, visuel opdatering og kommunikation med Arduinoen. De forskellige funktioner forklares i Tabellen nedenfor.

Funktion	Forklaring
sendCommand(string)	Ansvarlig for at opbygge og sende kommandoer gennem den serielle port efter den definerede kommunikationsprotokol.
displayFeedback(string)	Modtager Arduinoens datapakker, fortolker dem og opdaterer GUI'ens interne tilstand og visuelle elementer.
UpdateStatusLabel()	Opdaterer oversigten i GUI'en, så brugeren altid kan se gældende mode, lysintensitet og timerindstillinger.
GUI-navigation (on_buttonX_clicked)	Disse metoder håndterer skift mellem de forskellige sider i applikationen (velkomst, menu, automatisk mode, motion mode), når brugeren trykker på knapperne.
UpdatePowerButton()	Opdaterer udseende og tekst på tænd/sluk-knappen baseret på systemets aktuelle status.

Tabel 11 - GUI Klassediagram

Systemets softwaredesign bygger på en klar opdeling mellem brugergrænseflade, kommunikationslag og protokolhåndtering. GUI'en indsamler alle brugerens valg om indstillinger, som derefter omsættes til serielle datapakker baseret på den fastlagte kommunikationsprotokol. Denne protokol anvender enkle kommando-bogstaver efterfulgt af værdier, hvilket gør overførslen robust og let at tolke for Arduinoen.

GUI'en er implementeret i en enkelt klasse, der står for navigation mellem vinduer, visning af systemstatus og afsendelse af kommandoer. Når brugeren ændrer Mode, intensitet, Timer eller tilstand, konstruerer GUI'en den korrekte protokolstreng til Bluetooth-modulet, som sender den til Arduinoen. Omvendt modtager GUI'en systemfeedback, som tolkes og bruges til at opdatere visuelle elementer, så grænsefladen altid afspejler den reelle tilstand. Disse kommandoer gennemgås i dybden med i bilag 7 afsnit 3.6.6.

7.6.1 Kommunikationsprotokol mellem GUI→Bluetooth→Arduino

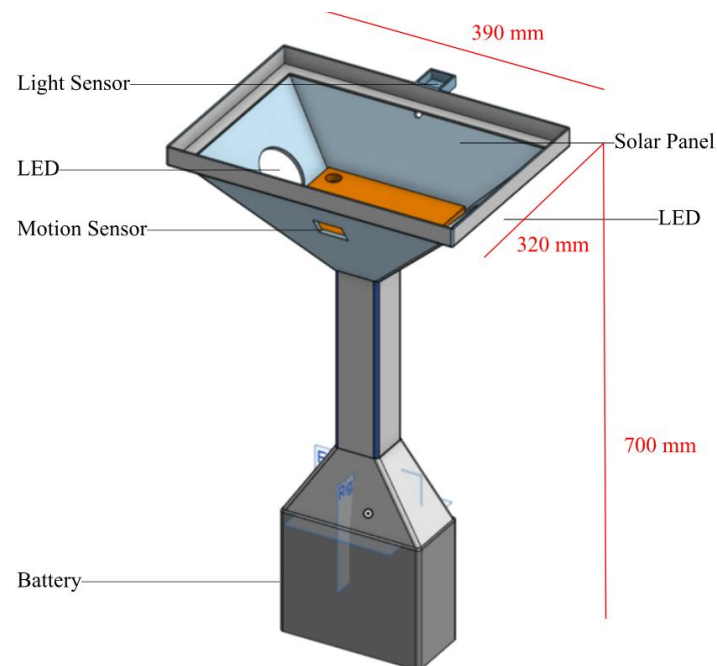
Kommunikationen er baseret på en protokol, hvor GUI'en sender datapakker gennem COM-porte med hjælp af Qt's metoder, defineret i bilag 7. Disse datapakker sendes til Arduinoen for at overføre brugerens indstillinger og kommandoer (defineret ud fra SON's Use Cases).

Kommando byte	Kommando	Værdi	Datatype
A	Vælg Mode	0: Automatic 1: Motion	String
B	Juster Lysintensitet	0-100 (LightIntensity i %)	String
C	Juster Timer	5-120 (sekunder)	String
D	Tænd/Sluk lampe	0: Sluk 1: Tænd	String
E	Anmodning om systemstatus	AX, BX, CX, DX	String

Tabel 12 – Kommandobeskrivelser

8. 3D printet kabinet

Med udgangspunkt i de udarbejdede hardwarekredsløb, og valgte hardwarekomponenter er der udarbejdet en 3D model i CAD-softwaren OnShape. Modellens udgangspunktet er solcellens dimensioner (397mm x 280mm), der udgør toppen af SON. Resten er sammensat med udgangspunkt i hardwarekomponenternes fysiske dimensioner, så designet understøtter funktionaliteten.



Figur 35 - CAD Model af systemet SON

Der er plads til solcellen øverst, LED'erne i siderne og en lyssensor bagerst samt en bevægelsessensor forrest, er det muligt at integrere næsten al hardwaren i solcelle lampens top. For at minimere støj og sikre korrekte målinger er lyssensoren placeret i lampens øverste del med fri adgang til omgivende lys. Batteriet er tungt, og fylder meget, og placeres derfor nederst i kabinettet. Denne placering af komponenterne gør at SON primært kan registrere bevægelse forfra, og særligt ideel langs en sti eller vej eller op ad en husmur. Da projektet udarbejdes som en prototype er kun lampens top 3D printet.

9. Modul- og integrationstests

I dette afsnit gennemføres modultests, der verificerer vores specificerede krav i Bilag 2 Tabel 1, med henblik på senere at kunne samle hele SON-systemet og verificere dets funktionalitet i en integrationstest. Ved dybdegående gennemgang henvises til bilag 8.

9.1 Modultest for PowerManagement

Formålet med testen er at verificere, at PowerManagement korrekt kan supplere SON med strøm, så LED'erne kan lyse, og lampen kan være solcelledrevet.

Som beskrevet i afsnit 3 af kravspecifikationen (bilag 2):

- **Krav 6** - SON skal kunne lyse med maksimal intensitet i 3 timer uafhængigt af vejret.
- **Krav 7** - Lampen skal være solcelle drevet med et batteri

Modultest for PowerManagement			
	Resultat	Test	Udfald
Strømtræk	Godkendt	Strømmen måles ved fuld LED-intensitet og sammenlignes med den teoretiske maksværdi på 1,6414 A.	Det målte strømtræk var lavere end 1,64 A, som forventet.
Solcelleforsyning	Godkendt	Udgangsspændingen solcellen måles med et voltmeter, og det kontrolleres, om den leverer spænding.	Solcellen leverer strøm, og kan supplere batteriet.

Tabel 13 - Modultest af PowerManagement

9.2 Modultest for System-klassen

Formålet med testen er at verificere følgende krav:

- **Krav 2** - Lampen skal have PWM-styring af lysintensiteten.
- **Krav 3** - Lampen skal kunne operere i Automatic Mode, der aktiveres af lysregistrering fra en lyssensor.
- **Krav 4** - Lampen skal kunne operere i Motion Mode, der aktiveres af bevægelsesregistrering fra en bevægelsessensor.

Modultest for System-klassen					
	LED	LightSensor	MotionSensor	Test	Resultat
1				Testet initialiserer systemet, opretter LEDDriver og ændrer intensiteten til 10, 20 og 0 med ét sekunds observation mellem hvert trin.	LED'erne ændrede intensitet tydeligt ved hver funktionskald.
2				Systemet tændes, LED'erne observeres i normal belysning, lyssensoren afskærmes for at sænke lysniveauet, systemet opdaterer målingen, og LED'erne observeres igen.	LED'erne tændte, når lyssensoren blev afskærmet, og gik inaktiv igen, når lyset vendte tilbage. Systemet reagerede stabilt og som forventet.
3				Systemet startes med korrekt indstillet PIR-sensitivitet; LED'erne kontrolleres uden bevægelse, testpersonen bevæger sig foran sensoren, MotionDriver registrerer aktiviteten og LED'ernes reaktion observeres.	LED'erne tændte tydeligt ved registreret bevægelse og forblev slukket inaktive uden aktivitet. PIR-sensoren reagerede stabilt og uden falske aktiveringer efter justering af sensitiviteten.

Tabel 14 - Modultest af System-klassen

9.3 Modultest for GUI

Formålet med testen er at verificere følgende krav.

Krav 5 - SON skal have en brugergrænseflade i flere niveauer, hvor lampens parametre kan ændres.

Modultest for GUI			
Funktion	Godkendt/ikke godkendt	Test	Resultat
Start-knap	Godkendt	Der klikkes på <i>Start</i> -knappen.	GUI'en skifter til hovedmenuen.
Motion Mode-knap	Godkendt	Der klikkes på Motion Mode-knappen.	GUI'en skifter til Motion Mode Menu.
Automatic Mode-knap	Godkendt	Der klikkes på Automatic Mode-knappen.	GUI'en skifter til Automatic Mode Menu.
Back-knap	Godkendt	Der klikkes på Back-knappen i en vilkårlig Mode menu.	GUI'en skifter til Hoved-menuen.
Timer-slider	Godkendt	Træk Timer-slideren i Motion Mode-menuen fra venstre mod højre og derefter fra højre mod venstre.	Terminalen viser værdier fra "C5" til "C120" og tilbage fra "C120" til "C5".
Light Intensity-slider	Godkendt	Træk Light Intensity-slideren fra venstre mod højre i Automatic Mode og fra højre mod venstre i Manuel Mode.	Terminalen viser værdier fra "B0" til "B100" og tilbage fra "B100" til "B0".
On/Off	Godkendt	Tryk på "Lights Off" og derefter "Lights On".	Terminalen viser "D1" og "D0", knappen skifter mellem "Lights On" og "Lights Off".

Tabel 15 - Modultest af GUI

9.4 Modultest for Bluetooth + GUI-Bluetooth-Kommunikation

Formålet med testen er at verificere følgende krav:

Krav 10 - Systemet kan kommunikere med GUI'en via Bluetooth.

Modultest for GUI-Bluetooth-Kommunikation			
	Godkendt/ikke godkendt	Test	Resultat
Ekko-test	Godkendt	Par HC-05-modulet med kontrolenheden og åbn en seriel terminal på den tildelte COM-port.	Arduinoen sender beskeden "Hello from Arduino! Ready to receive commands like A1 or A0.", hvilket bekræfter forbindelsen.
Parsing	Godkendt	Send tekststreng "A1" og "A0" fra master.	"A1" modtages og tænder LED'en, "A0" modtages og slukker LED'en.
GUI	Godkendt	Tryk på "Lights Off" og derefter "Lights On".	"D1" modtages og tænder LED'en, "D0" modtages og slukker LED'en.

Tabel 16 - Modultest af Bluetooth + GUI

9.5 Integrationstest

I dette afsnit udføres en integrationstest for SON, hvor de forskellige moduler sammensættes og testes for at sikre korrekt funktionalitet. Der tilføjes gradvist flere moduler for løbende at verificere den samlede systemintegration og sikre, at det fulde system fungerer efter hensigten. Den tager udgangspunkt i GUI'en forbundet til Bluetooth, lyssensoren og LED'en for at kunne eftervise funktionalitet.

Integrationstest for SON								
	GUI	Bluetooth	Light-Sensor	LED	Motion	PM	Test	Resultat
1							Start systemet, tilslut GUI via Bluetooth, skift til Automatic Mode, send lysintensitetskommando, og varier lyset foran lyssensoren.	Når lyssensoren registrerer dagslys, er LED'erne inaktive. Når lyssensoren afskærmes, tændes LED'erne med den intensitet, der er valgt i GUI'en.
2							Skift til Motion Mode i GUI, send lysintensitets- og Timerværdi, udfør bevægelse foran sensoren, og vent til Timeren udløber.	LED'erne er inaktive uden bevægelse, tænder ved registreret bevægelse med GUI-indstillet intensitet, og går inaktive igen automatisk efter 10 sekunder.
3							Tryk på On/Off-knappen i GUI med lyssensoren afskærmet.	LED'erne tænder ved On-knap og slukker ved Lights Off, mens GUI'en korrekt viser systemets feedback.
4							Testen af systemet med PowerManagement er udført på præcis samme måde, som testen uden PowerManagement.	Systemet bevarer alle funktioner og strøm løber fra PM.

10. Resultater

I dette kapitel vil resultaterne fra accepttesten blive vist. Resultaterne er baseret på den endelig accepttest som blev foretaget af Rasmus Vorup den 28/11/2025. Den fulde accepttest kan findes i bilag 2

10.1 Accepttest

Inden accepttesten blev vejlederen præsenteret for produktet uden forudgående instruktioner om dets anvendelse. Formålet med dette var at vurdere SON's brugervenlighed gennem en spontan og uforberedt interaktion.

Nedenfor præsenteres et udsnit af resultaterne fra accepttesten hvor # er antal trin i testen.

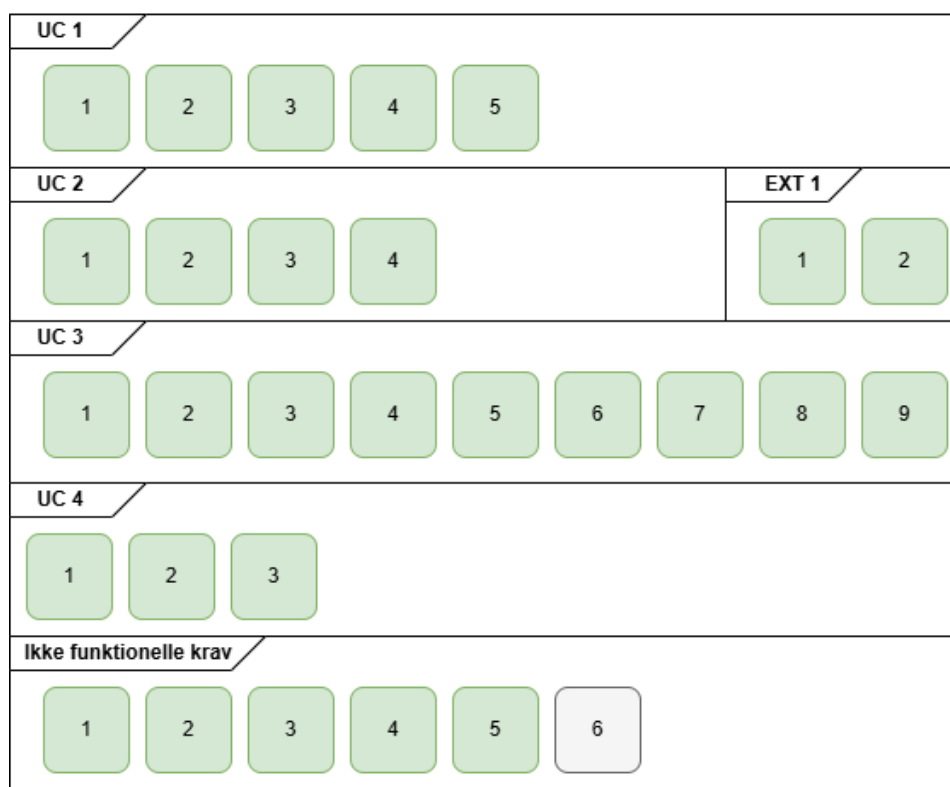
Funktionelle krav		
<i>Nr.</i>	<i>Vurdering</i>	<i>Kommentar</i>
UC1 #4 Choose Mode	Godkendt	Brugeren kan skifte imellem Modes.
UC2 #4 Adjust Light Intensity	Godkendt	Brugeren kan observere en tydelig ændring ved skift af lysintensitet
UC2 EXTENSION #2	Godkendt	Brugerens indstillinger huskes efter system genstart
UC3 #9 Adjust Timer	Godkendt	Lampen tænder efter det givne Timerinput af brugeren og overholder den accepterede latens
UC4 #3 Toggle Lamp	Godkendt	Lyset kan tændes og slukkes af brugeren
Ikke funktionelle krav		
<i>FURPS</i>	<i>Vurdering</i>	<i>Kommentar</i>
Functionality Krav 2	Godkendt	Systemet kunne huske indstillingerne efter GUI'en blev lukket ned. Derefter startede GUI'en op med samme indstillinger
Usability Krav 6	Ikke godkendt	Testpersonen kunne udføre og godkende alle accepttest uden instruktioner om anvendelse, dog blev en decideret test ikke foretaget.
Reliability Krav 3	Godkendt	Beregningerne fra bilag 4 viste, at SON kan lyse med

		maksimal styrke i op til tre timer på en fuld opladning.
Reliability Krav 5	Godkendt	Systemet vendte automatisk tilbage til den senest anvendte tilstand efter strømafbrydelse.
Performance Krav 1	Godkendt	Indstillingen til at justere lysintensitet havde en forsinkelse på under 1 sekund.
Supportability/Safety Krav 4	Godkendt	SON blev stress-testet ved, at testpersonen frit kunne afprøve alle tænkelige kombinationer for at identificere potentielle svagheder.

Tabel 18 - Tabel over accepttest

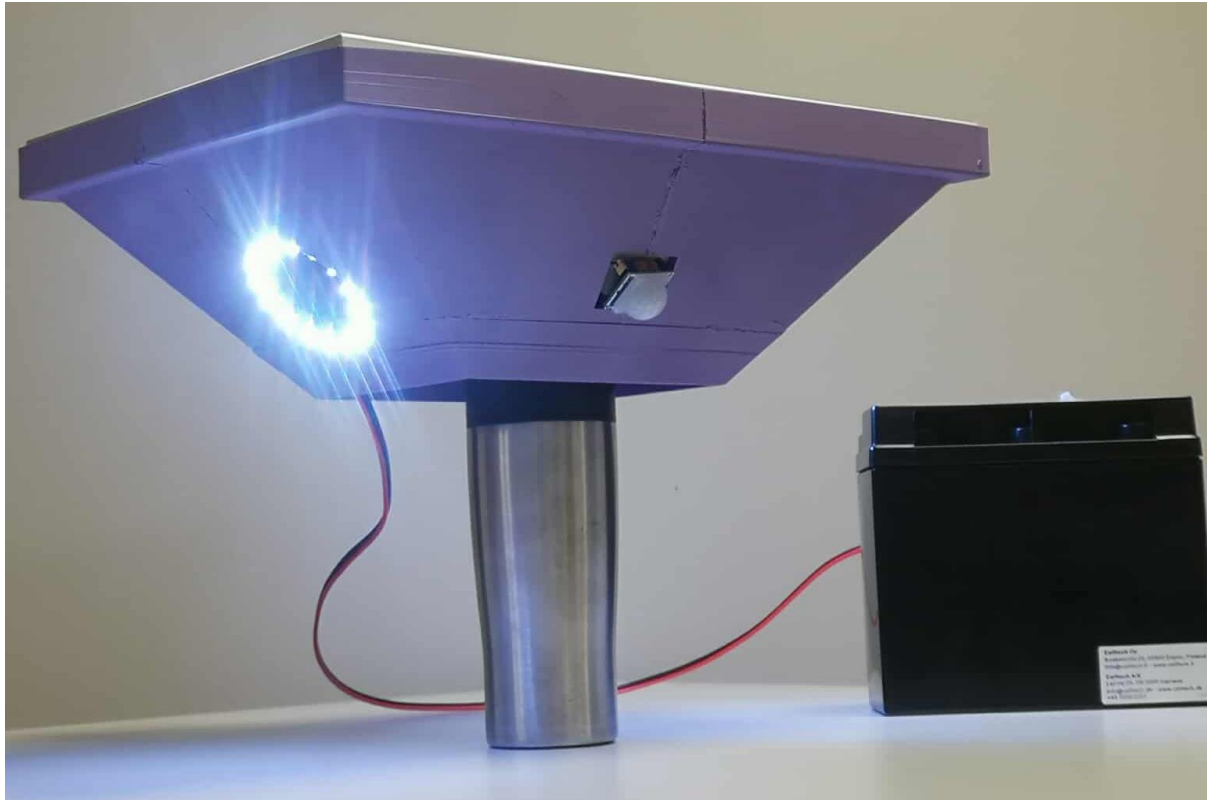
Tabellen ovenfor, er blot et udpluk af hele accepttesten som kan ses i bilag 2.

Et visuelt overblik er vist på nedenstående Figur. Grøn er godkendt, Rød ikke godkendt og grå ikke udført.



Figur 36 - Resultat af accepttest

Afslutningsvist ses nedenfor et billede af det færdige produkt ”SON”.



Figur 37 - Billede af det samlede produkt

11. Diskussion og fremtidigt arbejde

Alle opstillede test blev gennemført med succes, og SON levede op til både de funktionelle og ikke-funktionelle krav. Resultaterne viser, at lampen kan operere stabilt i både Automatic Mode og Motion Mode, og at PowerManagement fungerer som forventet.

En central del af udviklingen har været anvendelsen af AVR til implementeringen af størstedelen af koden. Selvom AVR er tidskrævende og oftest set som et komplekst sprog, så muliggør det adgang til en mere detaljeret kontrol over systemets funktioner og hardware.

Derudover er koden skrevet med en klasseorienteret struktur, hvilket betyder, at SON er modulært og nemt kan udvides. Det giver en fleksibel platform, hvor nye funktioner eller sensorer kan tilføjes uden større ændringer i den eksisterende kodebase.

Implementeringen af Bluetooth-kommunikation udgør et væsentligt resultat i projektet. Kommunikationsdelen blev implementeret på trods af begrænset erfaring med protokoller af denne type. Bluetooth forbindelsen gør SON til en mere funktionel prototype, idet alternativet med en konfigurerbar solcellelampe med kabelføring ville gøre GUI interfacet unødvendigt og SON mindre brugervenligt.

Den udviklede kodebase fremstår som stabil og velstruktureret. Ensartede konventioner for navngivning og opbygning har bidraget til høj læsbarhed og vedligeholdelsesvenlighed. Den klare struktur har desuden understøttet en effektiv integration mellem hardware og software, hvilket har reduceret risikoen for fejl i implementeringsfasen.

SON har demonstreret robusthed under stresstest og tænd/sluk-cykler. Funktionaliteten forblev intakt efter disse afbrydelser, hvilket indikerer en høj grad af stabilitet i både kode og systemarkitektur. Denne observation styrker vurderingen af, at SON er en solcelledrevet lampe, der kombinerer energieffektivitet og fleksibel brugerinteraktion uden hyppige nedbrud eller fejl.

SON er bygget som proof of concept med fokus på dokumentation, sporbarhed og gennemsigtighed, så SON's opbygning kan forklares, forsvares og videreudvikles, men der er potentiale for forbedringer. Grundlaget for komponentvalget var i høj grad bestemt af tilgængelighed, pris og implementeringsmuligheder. Ved fremtidigt arbejde ville det være at fortrække at anvende mere produktorienterede komponenter såsom et mindre batteri og mindre

solcelle. Dette ville gøre projektet billigere og tilnærme sig et reelt kommercielt produkt, der kunne konkurrere med lignende alternativer. Arduinoen kunne også udskiftes med en mere specialiseret styringsenhed, der kan håndtere systemets funktionalitet, med mindre strømforbrug, lavere pris og mindre fysiske dimensioner. Ligeledes kunne 3D printet udskiftes af et mere optimeret kabinet i mere optimale materialer, der underbygger moderne design i både form og funktionalitet.

12. Konklusion

Formålet med projektet var at designe og udvikle en solcelledrevet lampe, der kombinerer energieffektivitet med fleksibel brugerinteraktion. På baggrund af det færdige system og de gennemførte tests kan det konkluderes, at denne målsætning er opfyldt.

Energieffektiviteten er opnået gennem et PowerManagement kredsløb, der styrer opladning og afladning af batteriet og muliggør drift uafhængigt af ekstern strømforsyning. Systemet registrerer omgivelsernes lysniveau via lyssensoren og kan dermed automatisere oplysning i Automatic Mode, så lampen kun er aktiv, når det er nødvendigt. I kombination med kravene til maksimal driftstid ved fuld lysintensitet viser både beregninger og målinger, at systemet udnytter den tilgængelige energi hensigtsmæssigt.

Kravet om en bevægelsesbaseret driftsmode er realiseret gennem Motion Mode, hvor lampen tændes ved registrering af bevægelse og forbliver aktiv i en tidsstyret periode. Dette understøtter idéen om, at systemet skal reagere på brugerens ankomst og afrejse og dermed undgå unødvendigt energiforbrug, når området er tomt.

Den fleksible brugerinteraktion er realiseret gennem en GUI, hvor brugeren kan justere lysintensitet, vælge mellem Automatic og Motion Mode samt konfigurere timer-indstillinger. Integrationstesten viser, at kommunikationen mellem GUI, mikrocontroller og sensorer fungerer stabilt, og at ændringer på GUI'en afspejles på lampen i realtid inden for de opstillede krav til responstid.

Samlet set reducerer SON afhængigheden af ekstern strømforsyning og unødig kabelføring, samtidig med at systemet tilbyder større fleksibilitet end en traditionel, fast indstillet udendørslampe. Projektet resulterer således i et velfungerende proof-of-concept på en solcelledrevet lampe, der både optimerer energiforbrug og giver brugeren høj grad af kontrol over systemets adfærd - i overensstemmelse med problemformuleringen.

13. Individuelle konklusioner

Viktor

I dette projekt har jeg primært arbejdet med udviklingen af Bluetooth-kommunikation mellem mikrocontrolleren og GUI'en. Arbejdet har både været lærerigt, spændende og det har også givet mig praktisk erfaring med seriel kommunikation. Det har også styrket min forståelse af samspillet imellem hardware og software.

Udviklings- og implementeringsfasen har jeg derfor oplevet som overskuelig og relativt lettilgængelig.

De største udfordringer lå for mig i projektets tidligste faser, særligt udarbejdelsen af en passende accepttest og use cases. Det var udfordrende at definere tests for et system, der endnu ikke var udviklet, selvom jeg forstår deres betydning for at strukturere det endelige produkt. Samlet set har projektet givet mig vigtig erfaring især indenfor UART, en bedre forståelse af projektplanlægning og udarbejde men også arbejds punkter som jeg vil udvikle mig på til næste semesters projekt.

Alexander

I dette projekt har jeg arbejdet bredt på tværs af både de fælles områder og de mere tekniske dele af systemet, herunder udviklingen af GUI'en, systemkoden og dele af Bluetooth-kommunikationen. Gennem arbejdet er jeg blevet markant bedre til at strukturere et større softwareprojekt og har fået erfaring med, hvordan tidlige faser som kravspecifikation, use cases og arkitektur kan identificere problemer, før de opstår i implementeringen.

En af projektets udfordringer var at fordele tiden hensigtsmæssigt, da vi i starten hverken kendte projektets fulde omfang eller hvor lang tid opgaverne ville tage. Det betød, at vi løbende måtte gennemtænke planen og overveje at fravælge enkelte idéer for at holde os inden for tidsrammen. Dette har givet mig værdifuld erfaring i realistisk planlægning og prioritering.

Gustav

I dette projekt har jeg arbejdet med brugergrænsefladen og i den forbindelse udviklet en GUI til SON. Det har været en meget lærerig proces at arbejde med både god funktionalitet og en intuitiv brugeroplevelse. Derudover har jeg arbejdet med et for mig nyt værktøj: Qt, som er

optimeret til at udvikle GUI'er til forskellige systemer. Det var spændende at arbejde med dette bibliotek og optimere de rette funktionkald, så de stemte overens med den kommunikationsprotokol, vi havde designet til GUI-Bluetooth-kommunikationen.

Desuden har dette semesterprojekt været særligt nyt i forhold til måden, vi har dokumenteret projektet på, da der har været skærpede krav til rapporten. Det har været en lærerig proces at udarbejde så meget forarbejde, design og kravspecifikation inden implementeringen, samt at gennemføre tests og efterprøvelse. Forarbejdet har tydeligt gjort selve implementeringen meget nemmere, og den erfaring vil jeg tage med videre i fremtidige projekter.

Malthe

I dette projekt har mit primære udbytte været at arbejde med et selvvalgt projekt fra start til slut og opnå erfaring med at udarbejde den nødvendige dokumentation helt fra bunden. At projektet var selvvalgt har skabt en stærkere følelse af ansvar og medejerskab, hvilket har bidraget positivt til gruppens arbejdsproces.

Mit hovedansvarsområde har været inden for hardware og integrationen af de forskellige komponenter. Dette arbejde har været særligt lærerigt, da det har styrket og cementeret den viden, jeg tilegnede mig på første semester samtidig med at det gav praktisk erfaring med at koble teoretiske principper til konkrete implementeringer.

Samlet set har projektet givet mig en dybere forståelse af hele udviklingsprocessen og de krav, der følger med at udarbejde et komplet system. Jeg føler mig derfor langt bedre rustet til at tackle kommende semesterprojekter og til at arbejde mere struktureret og målrettet med både dokumentation, design og implementering.

Mads

Projektet har været særligt spændende, da den øgede selvstændighed og stillingtagen i forhold til at definere, hvad og hvordan vi ville udvikle, har været en motiverende faktor. Dette var et helt nyt tiltag til vores måde at lave projektarbejde og gav virkelig et nyt indblik i hvad det kræver at opbygge noget nyt fra bunden. Der skulle laves mere research, end hvad vi før havde været vant til, som gjorde processen mere spændende.

Jeg har primært arbejdet med udviklingen af Bluetooth-kommunikation mellem microcontrolleren og GUI'en. Dette har været lærerigt, givet mig praktisk erfaring med seriel kommunikation og styrket min forståelse af samspillet imellem hardware og software.

Det mest udfordrende for mig må være i projektets tidligste faser, hvor særligt udarbejdelsen af en passende accepttest og use cases for et system, der endnu ikke var udviklet kunne være indviklet. Dette er et klart arbejds punkt for fremtidige projekter. Samlet set har projektet givet mig vigtig teknisk erfaring og en bedre forståelse af projektarbejdsprocessen.

Oskar

Projektet har været nyt for mig, da tilgangen til arbejdet har været meget struktureret, og det har tydeligt vist, hvor vigtigt det er at have en klar rød tråd fra kravspecifikation til design og videre til implementering. Ved løbende at dokumentere vores valg er det blevet nemmere at skabe overblik og forstå, hvorfor bestemte løsninger er valgt, og hvordan de hænger sammen med projektets mål.

Mit primære fokus i projektet har været systemets hardwarekomponenter og samspillet mellem de enkelte dele. Det har været meget motiverende at arbejde med et fysisk produkt og se, hvordan systemet gradvist tog form.

Samlet set har projektet givet mig værdifuld erfaring med både projektarbejde og dokumentation, som jeg vil tage med videre og arbejde endnu mere bevidst med i kommende projekter.

14. Bilagsliste

1. Bilag/Projekt/01_Problemformulering.pdf
2. Bilag/Projekt/02_Kravspecifikation og Accepttestspecifikation.pdf
3. Bilag/Proces/03_Metode og Proces.pdf
 - 3.1 Bilag/Proces/03.1_Mødedeltagelse.pdf
 - 3.2 SW8PRJ2/Proces/03.2_Tidsplan.pdf
 - 3.3 SW8PRJ2/Proces/03.3_Samarbejdskontrakt.pdf
4. Bilag/Projekt/04_Teknisk Analyse.pdf
5. Bilag/Projekt/05_Systemarkitektur.pdf
6. Bilag/Projekt/06_Navneordanalyse.pdf
7. Bilag/Projekt/07_Design og Implementering.pdf
8. Bilag/Projekt/08_Modultest og Integrationstest.pdf
9. Bilag/Projekt/09_Sourcekode/
10. Bilag/Projekt/10_Datablade/
11. Bilag/Proces/11_Vejledermøder/
12. Bilag/Proces/12_AI-Deklaration/

15. Bibliografi

1, I. L. (u.d.). Hentet fra <https://brightspace.au.dk/d2l/le/lessons/183300/units/2312181>

Pinout, M. A. (u.d.). Hentet fra
<https://brightspace.au.dk/d2l/le/lessons/165648/units/2121829>

Garric, A. (2025, July 25). *Climate: What does it mean if we can no longer limit warming to 1.5 °C?* *Le Monde*. https://www.lemonde.fr/en/environment/article/2025/07/25/climate-what-does-it-mean-if-we-can-no-longer-limit-warming-to-1-5-c_6743702_114.html